



Accelerated Quantum Supercomputing

Hour 2: Generative Quantum Eigensolver (GQE)

Pooja Rao (NVIDIA)

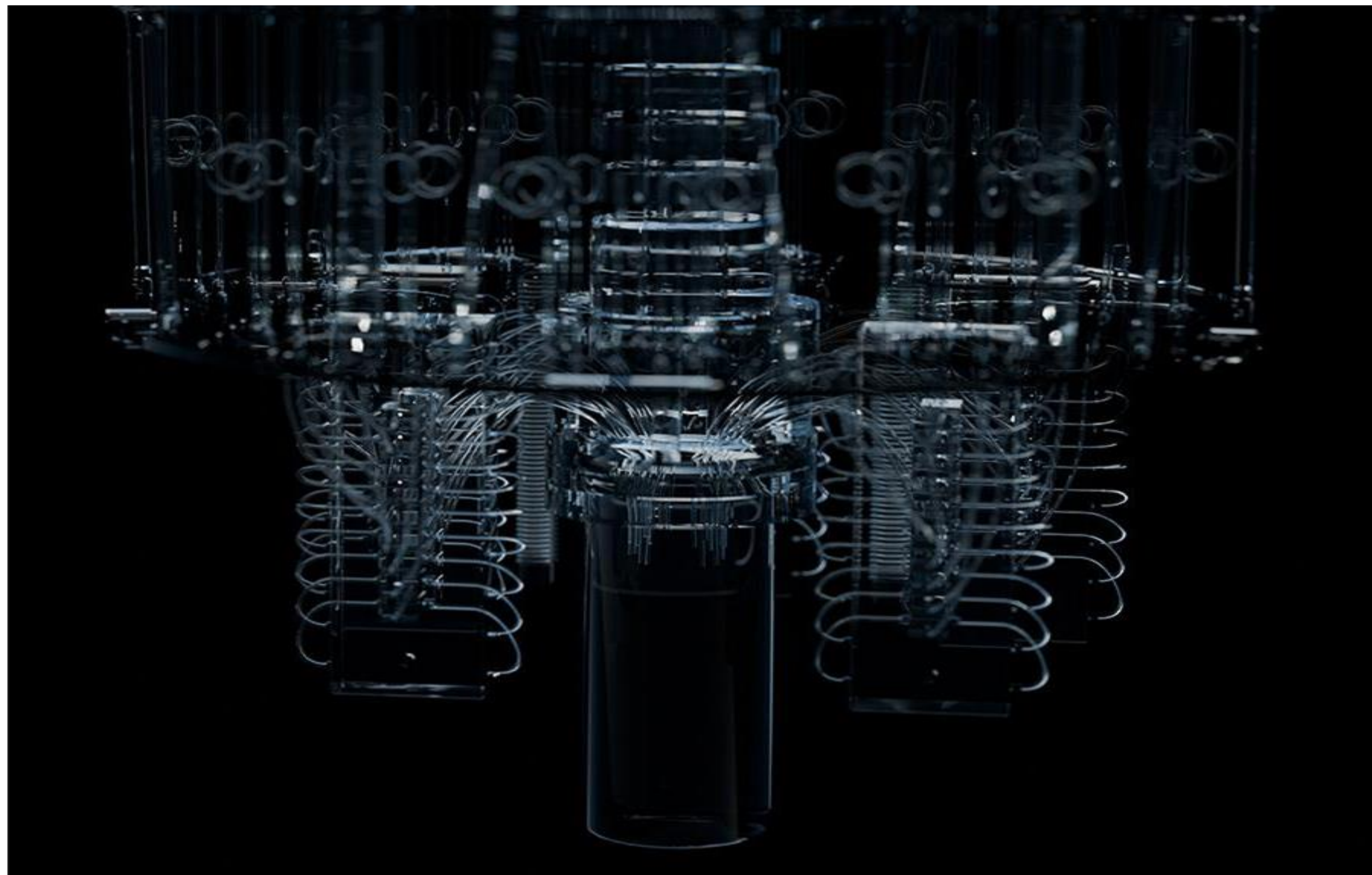
Senior Quantum Algorithm Engineer

SC25, Nov 16, 2025

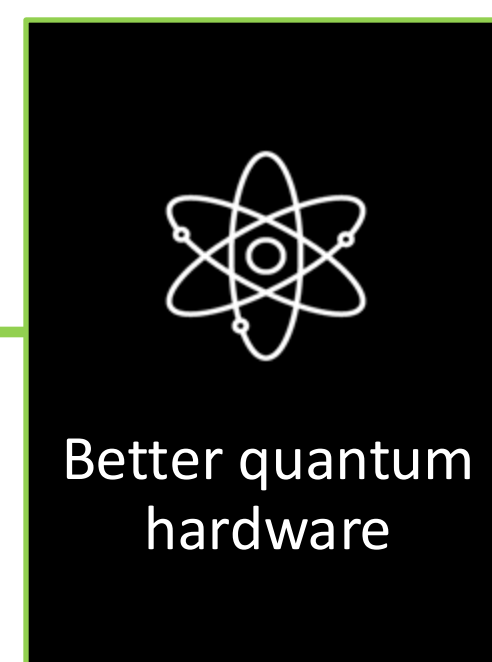
Outline

- Overview of AI for Quantum + CUDA-Q
- GQE vs VQE
- Analogy between text generation and circuit generation
- Benchmarks on simulator
- VQE vs GQE benchmarking
- Improvement with GRPO as a cost function
- Parallelization
- Benchmark on a quantum hardware
- Outlook: Transfer learning, pre-training, offline training, etc.

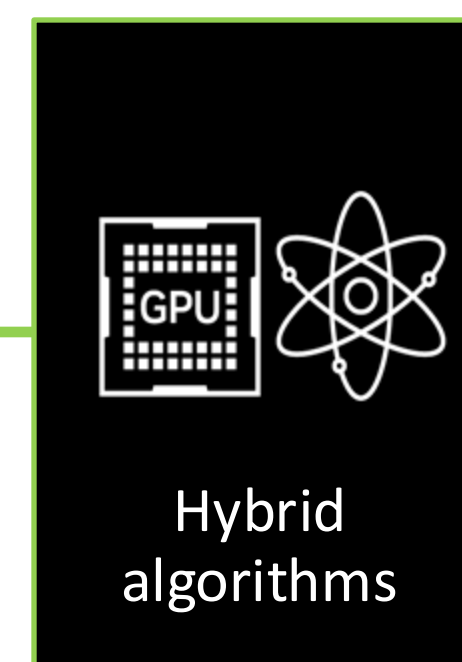
Accelerating the Journey From Qubits to Supercomputers



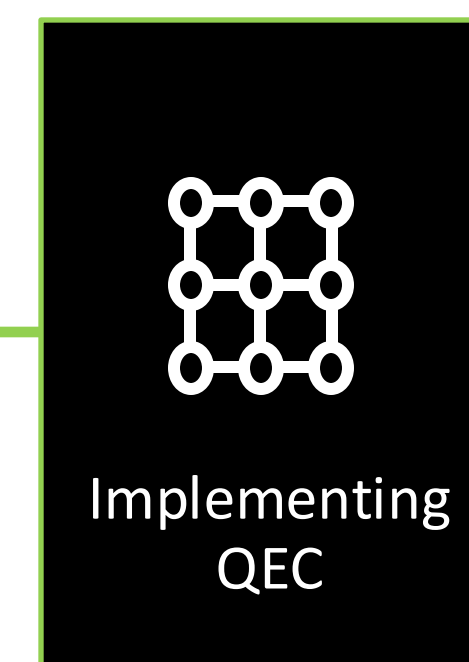
Qubits



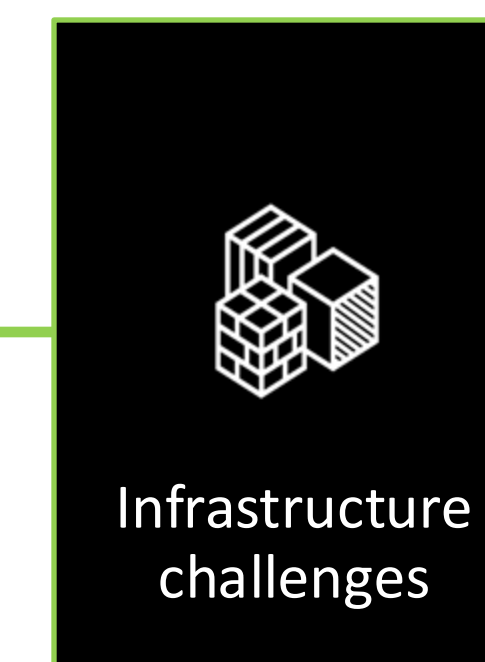
Better quantum hardware



Hybrid algorithms

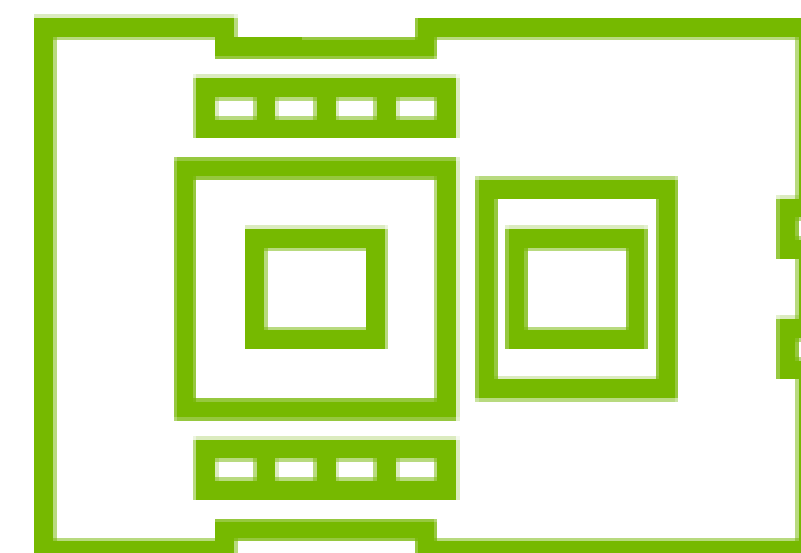


Implementing QEC



Infrastructure challenges

Accelerated Quantum Supercomputers



AI supercomputing

CUDA-Q

The platform for accelerated quantum computing

Python and C++

Access via familiar & powerful languages

QPU agnostic

Supports backends from all major QPU vendors and qubit types

GPU-accelerated

Quantum simulators that scale to large-scale quantum computers

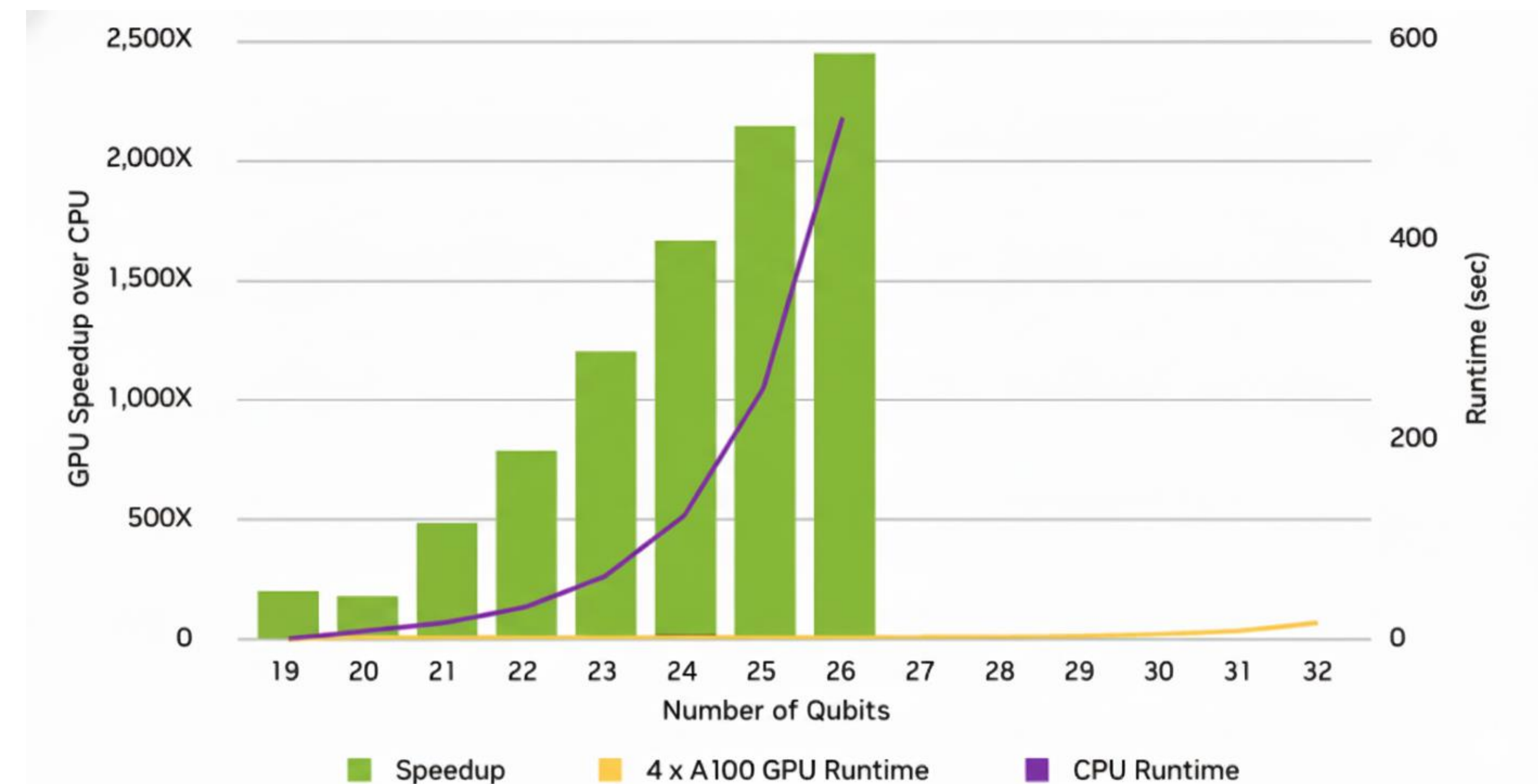
Fully **kernel system for hybrid computing interface**

Seamlessly combine GPU and QPU resources

Access to classical **CUDA-X libraries**

Conventional parts of hybrid algorithms can draw on fastest implementations

Performance



QML workflow in CUDA-Q using **multithreaded CPU** versus **NVIDIA A100 GPUs**

CUDA-Q Overview

<https://developer.nvidia.com/cuda-q>

CUDA-Q Academic

<https://github.com/NVIDIA/cuda-q-academic>

CUDA-Q Docs

<https://nvidia.github.io/cuda-quantum/latest/index.html>

CUDA-Q Apps

<https://nvidia.github.io/cuda-quantum/latest/using/tutorials.html>

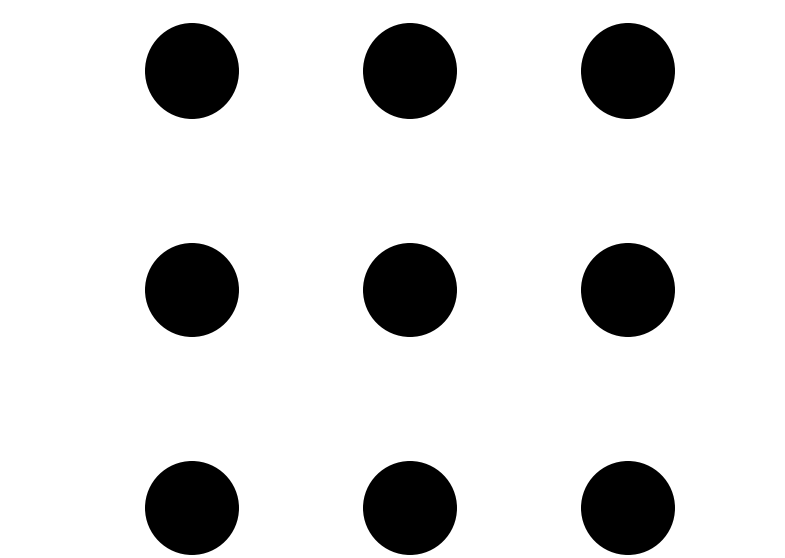
CUDA-Q Libraries

Bringing the power of CUDA-Q directly to quantum computing research problems

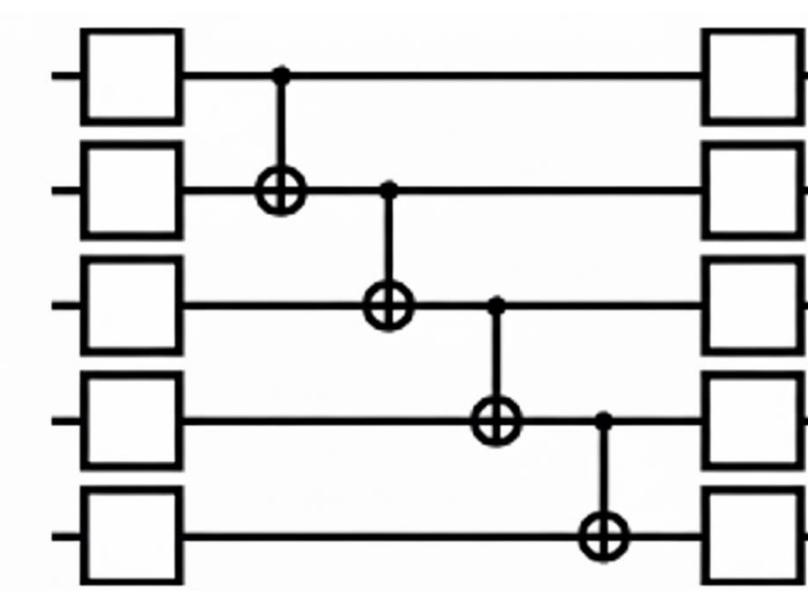


NVIDIA CUDA-QX

Seamless integration of
accelerated **QEC primitives**
into CUDA-Q workflows



CUDA-Q QEC



CUDA-Q Solvers

Accelerating **solvers** – key
algorithms within many
quantum applications

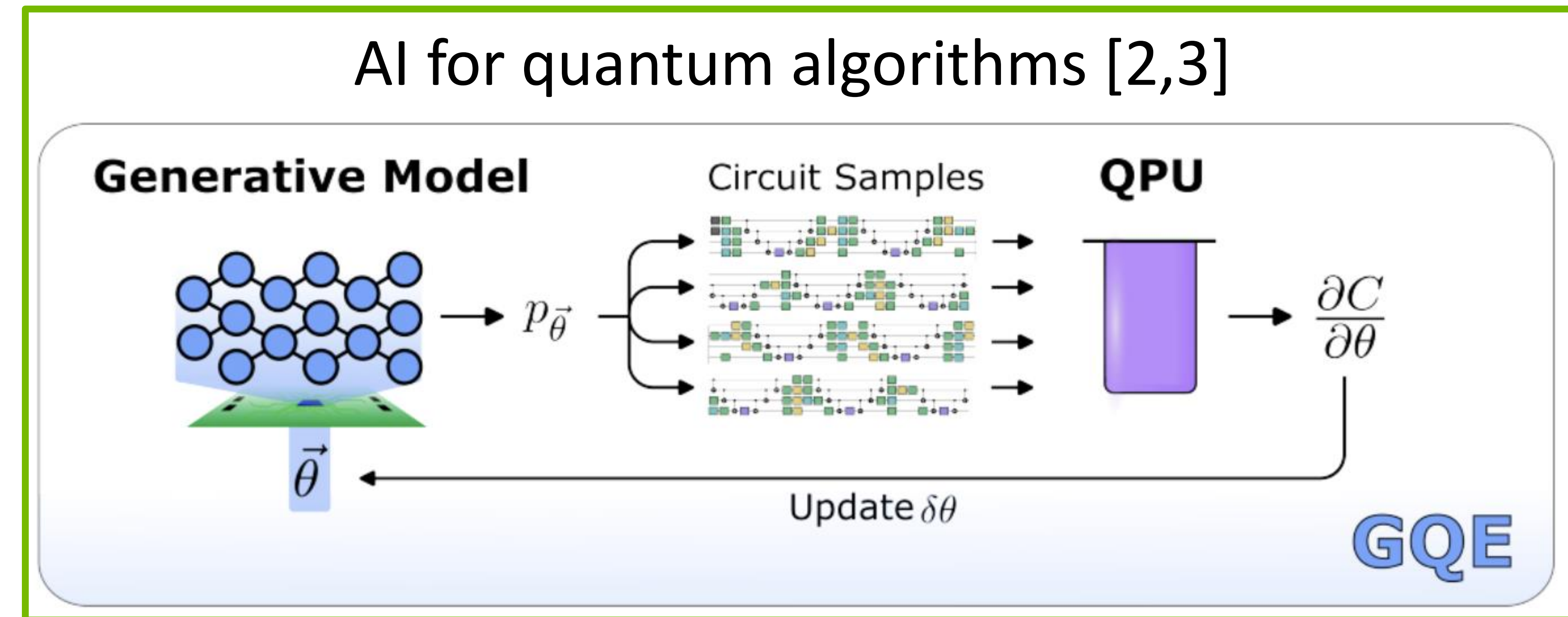
NVIDIA CUDA-Q

Accelerated Quantum Supercomputing

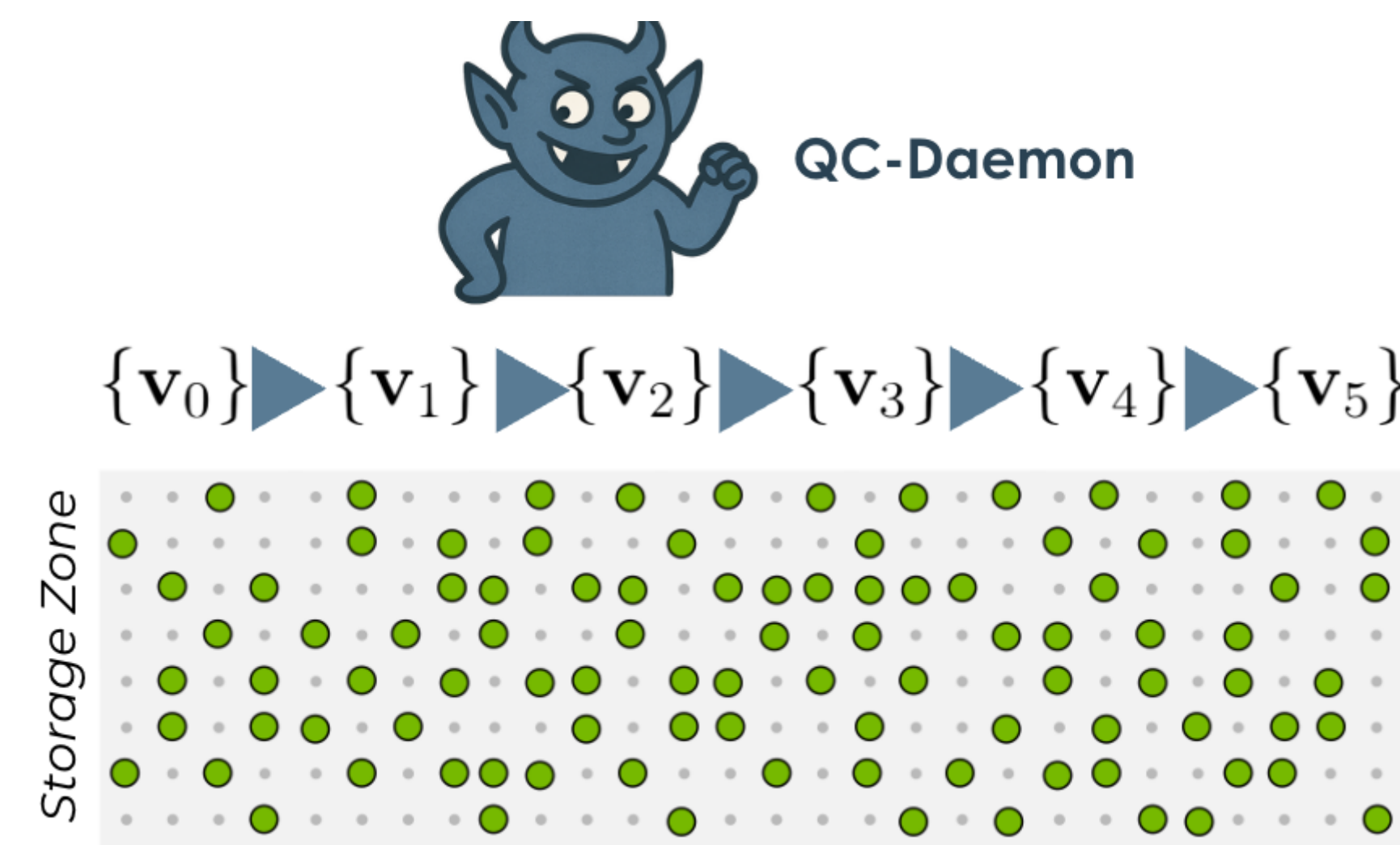
GQE Code



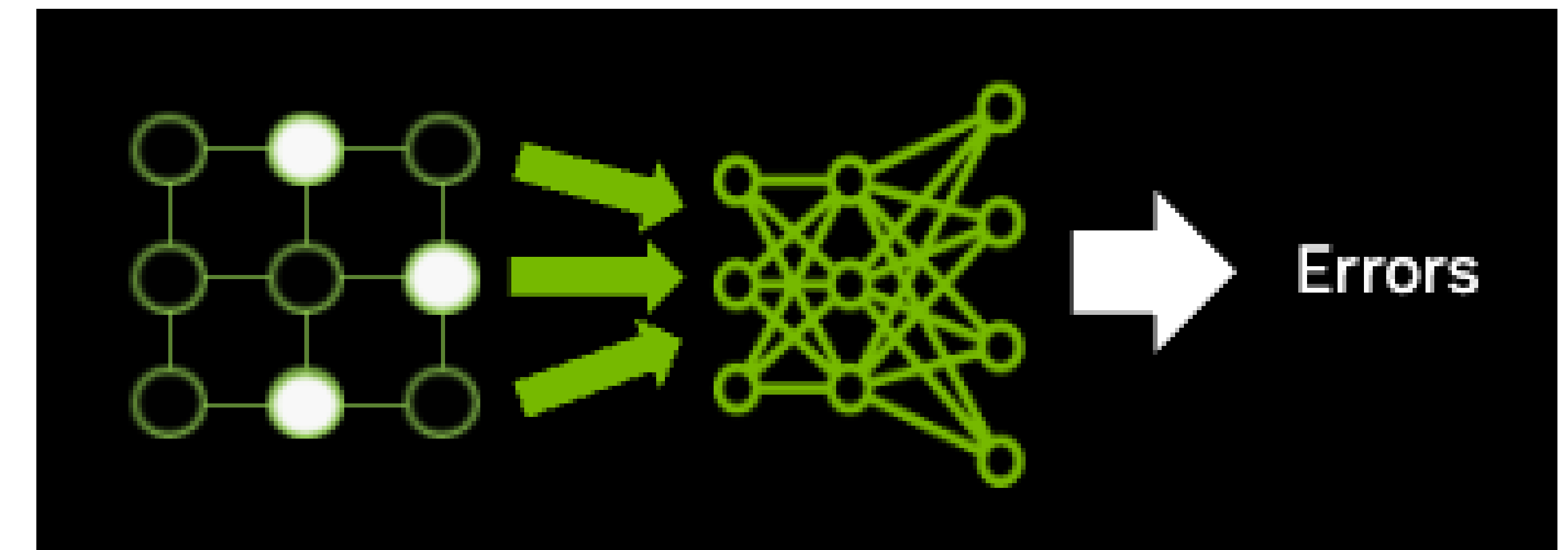
AI for quantum computing [1]



AI for quantum device control [4]



AI for quantum error correction [5]



[1] Alexeev, Yuri, et al. "Artificial Intelligence for Quantum Computing." arXiv preprint arXiv:2411.09131 (2024).

[2] **K. N.**, et al. "The generative quantum eigensolver (GQE) and its application for ground state search." *arXiv:2401.09253*.

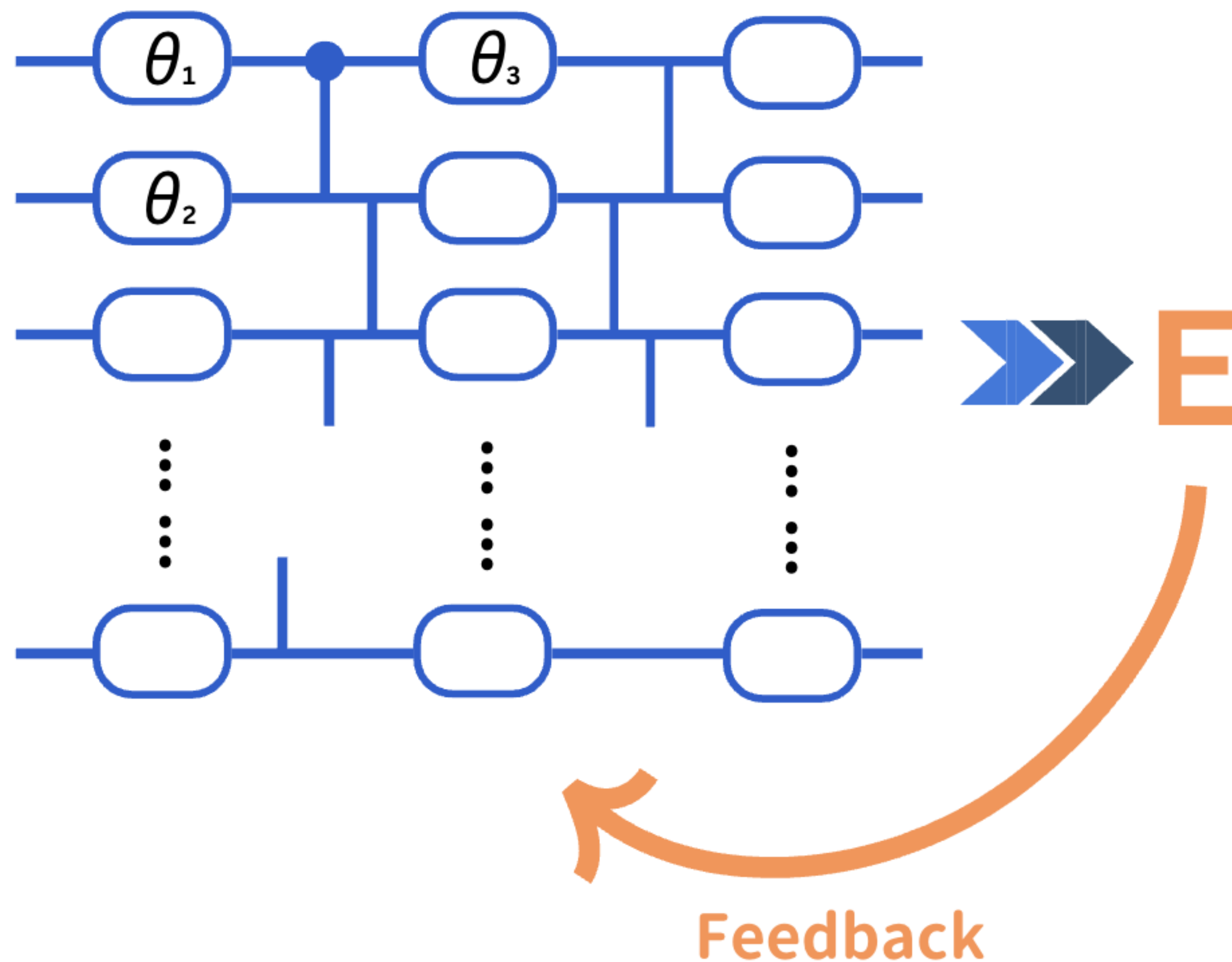
[3] S. Minami, **K. N.**, Y. Suzuki, A. Aspuru-Guzik & T. Kadowaki "Generative quantum combinatorial optimization by means of a novel conditional generative quantum eigensolver." *Digital Discovery*, 2025, 4, 2229-2243

[4] **K. N.**, et al. Quantum circuits as a game: A reinforcement learning agent for quantum compilation and its application to reconfigurable neutral atom arrays. *arXiv:2506.05536*

[5] H. Zhou, C. Zhao, **K.N.**, J. Lietz, A. McCaskey, T. Patti, and C. Chamberland, "NVIDIA and QuEra Decode Quantum Errors with AI" (NVIDIA blog)

Variational quantum algorithm (VQA)

As a near-term algorithm, VQA was proposed a decade ago from my previous (Aspuru-Guzik) group, and has been broadly studied in the community



- Very actively investigated in this decade.
- The optimization becomes more and more challenging in larger problems.
- Many measurements are required
- Efficiently loading classical data is challenging

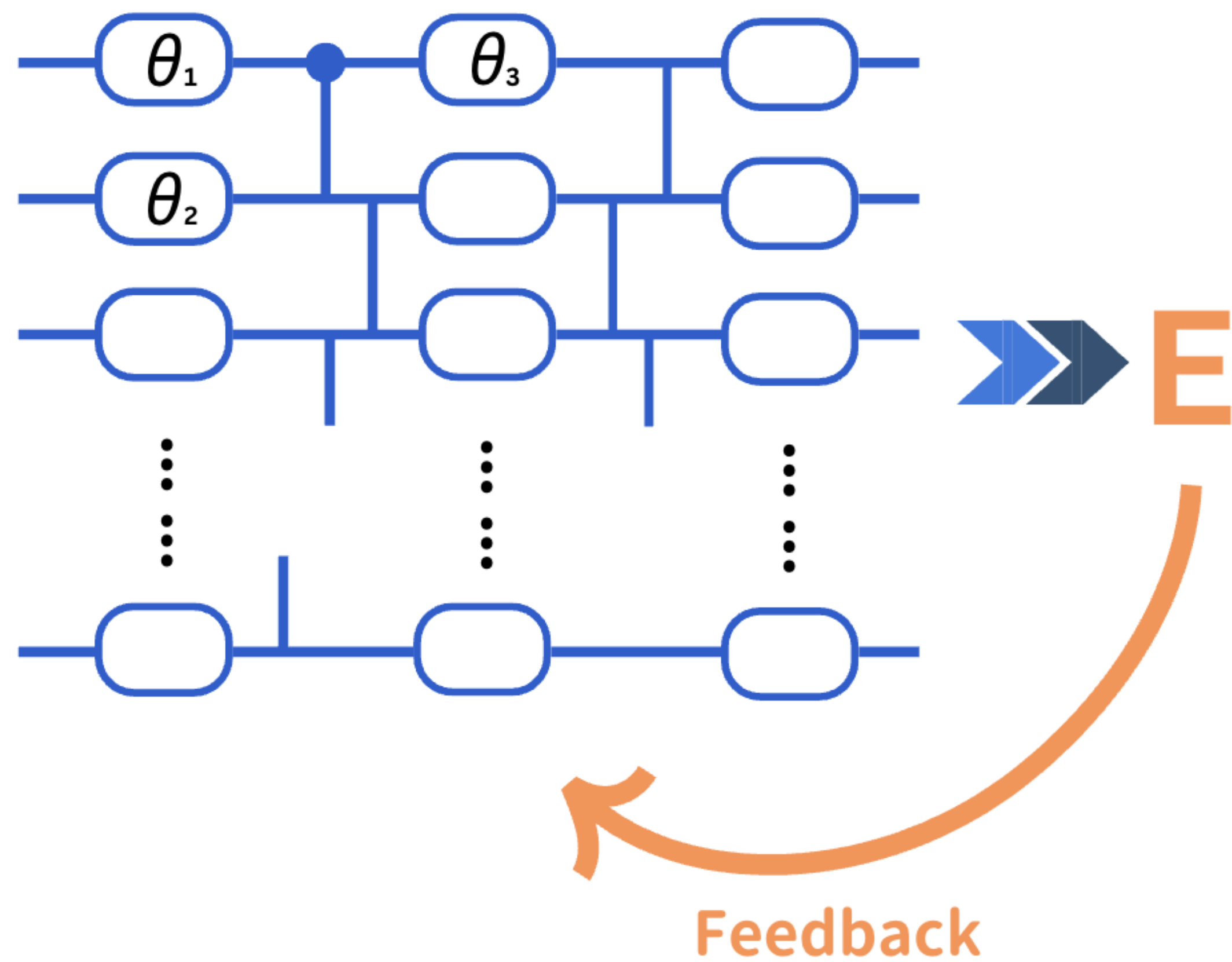
P. Alberto, J. McClean, P. Shadbolt, M. Yung, X. Zhou, P. J. Love, A. Aspuru-Guzik, and J. L. O'Brien. *Nature communications* 5, no. 1 (2014): 1-7.

**Is it possible to create a new framework,
Which is orthogonal to VQA?**

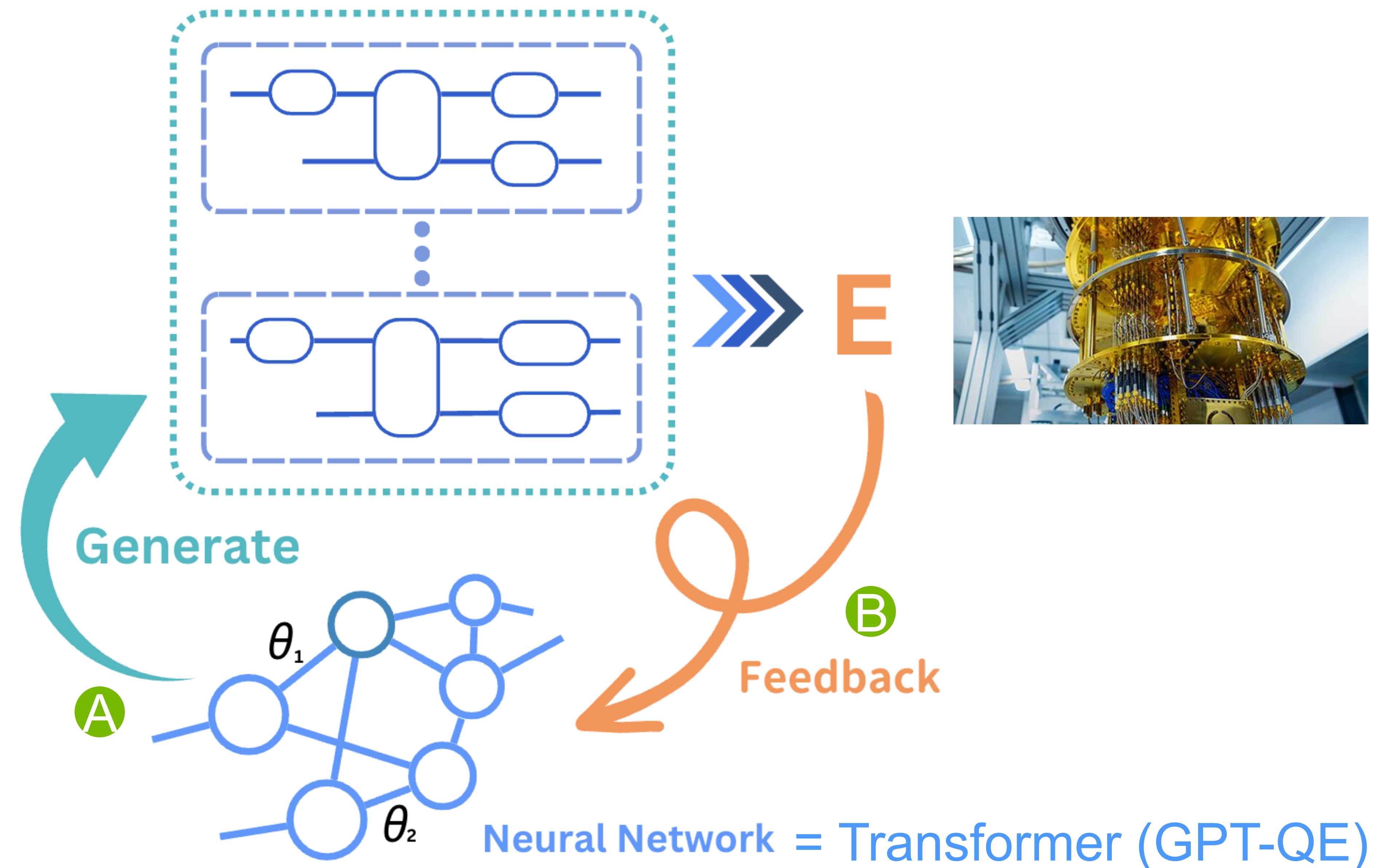
Our concept: generative quantum eigensolver (GQE)

GQE uses the generative model of quantum circuits.

VQE



Generative quantum eigensolver (GQE)

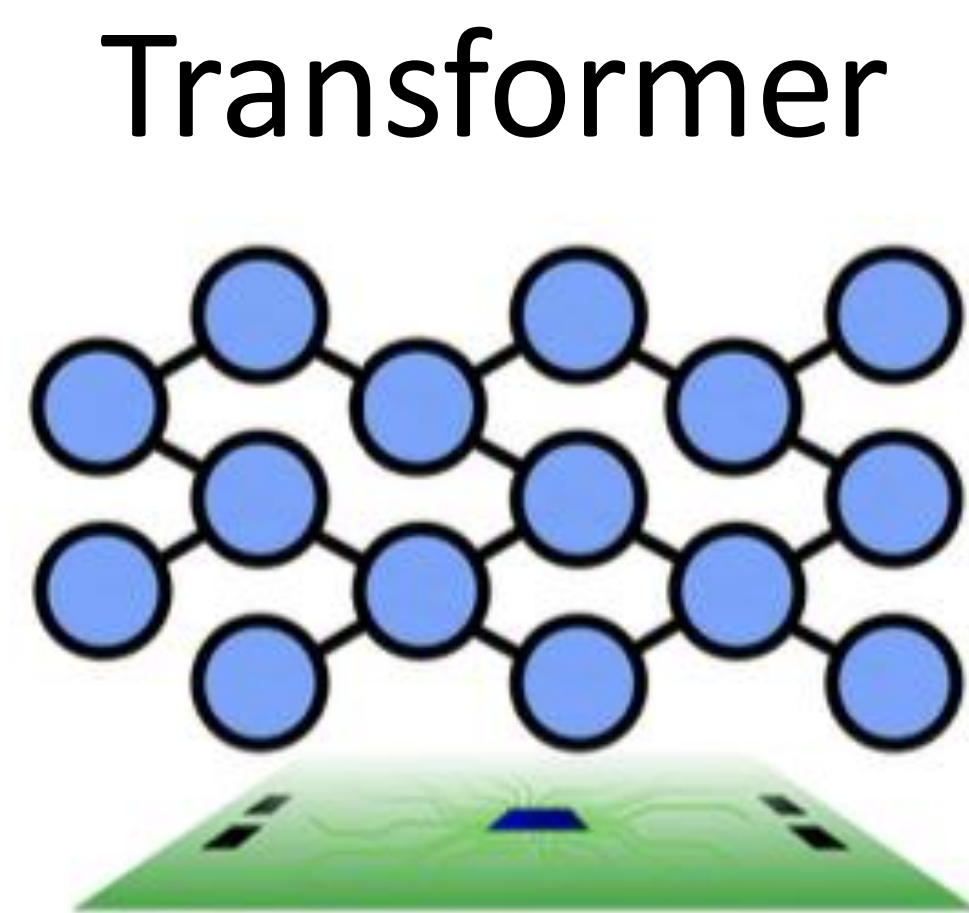


[1] K. Nakaji, et al. "The generative quantum eigensolver (GQE) and its application for ground state search." *arXiv preprint arXiv:2401.09253* (2024).

A

How can we generate quantum circuits?

Each generated token is mapped to a quantum circuit component



Tokens

$$\vec{j} = \{j_1, \dots, j_N\} \quad 1 \leq j_1, \dots, j_N \leq L$$

$$j_1 \leftrightarrow \text{word}_{j_1}$$

$$\vec{j} \sim \begin{matrix} \text{word}_{j_1} & \text{word}_{j_2} & \text{word}_{j_3} & \text{word}_{j_4} & \dots & \text{word}_{j_{N-2}} & \text{word}_{j_{N-1}} & \text{word}_{j_N} \end{matrix} \quad \text{LLM}$$

$$j_1 \leftrightarrow U_{j_1}$$

$$\vec{j} \sim \begin{matrix} U_{j_1} & U_{j_2} & U_{j_3} & \dots & U_{j_N} \end{matrix} \quad U(\vec{j}) \quad \text{GPT-QE}$$

Operator pool: $\{U_1 \dots U_L\}$

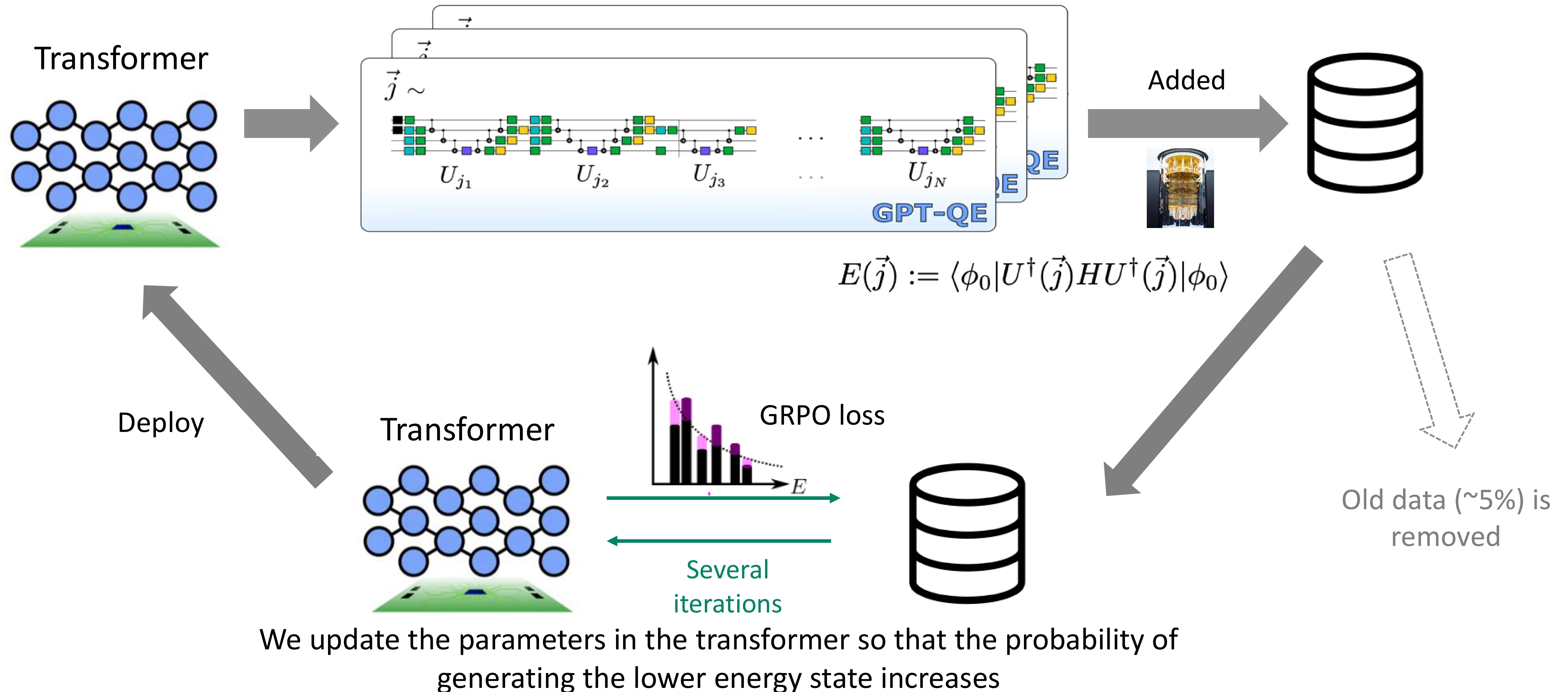


$$E(\vec{j}) := \langle \phi_0 | U^\dagger(\vec{j}) H U(\vec{j}) | \phi_0 \rangle \quad H: \text{An observable (e.g. chemistry Hamiltonian)}$$

B

How can we train the model?

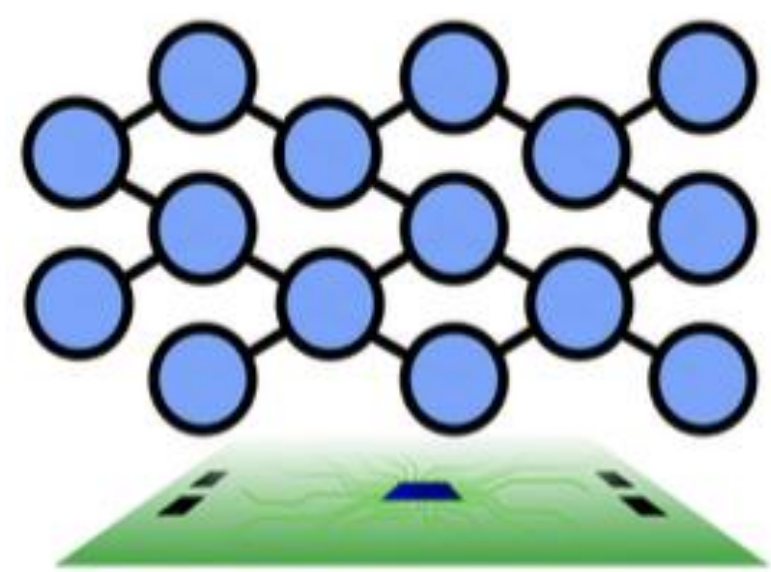
We train the model with the dataset generated



Proof-of-concept: GPT-QE with chemistry Hamiltonians

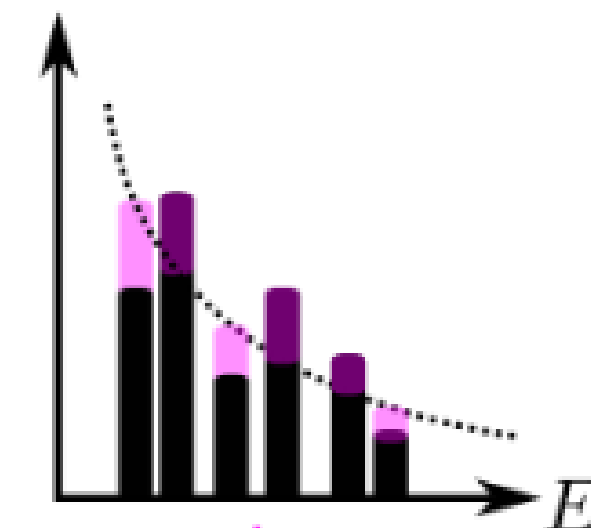
Workflow

Transformer



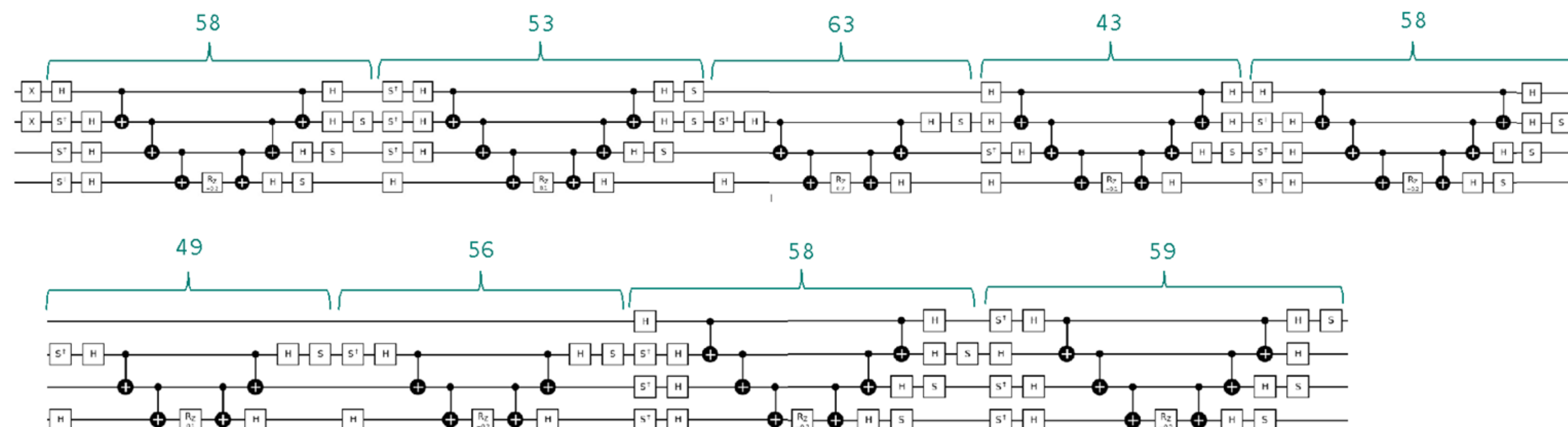
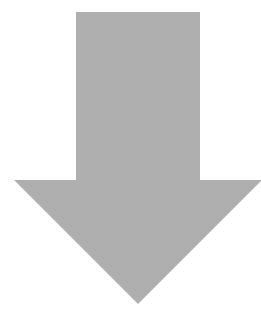
training

Simulator



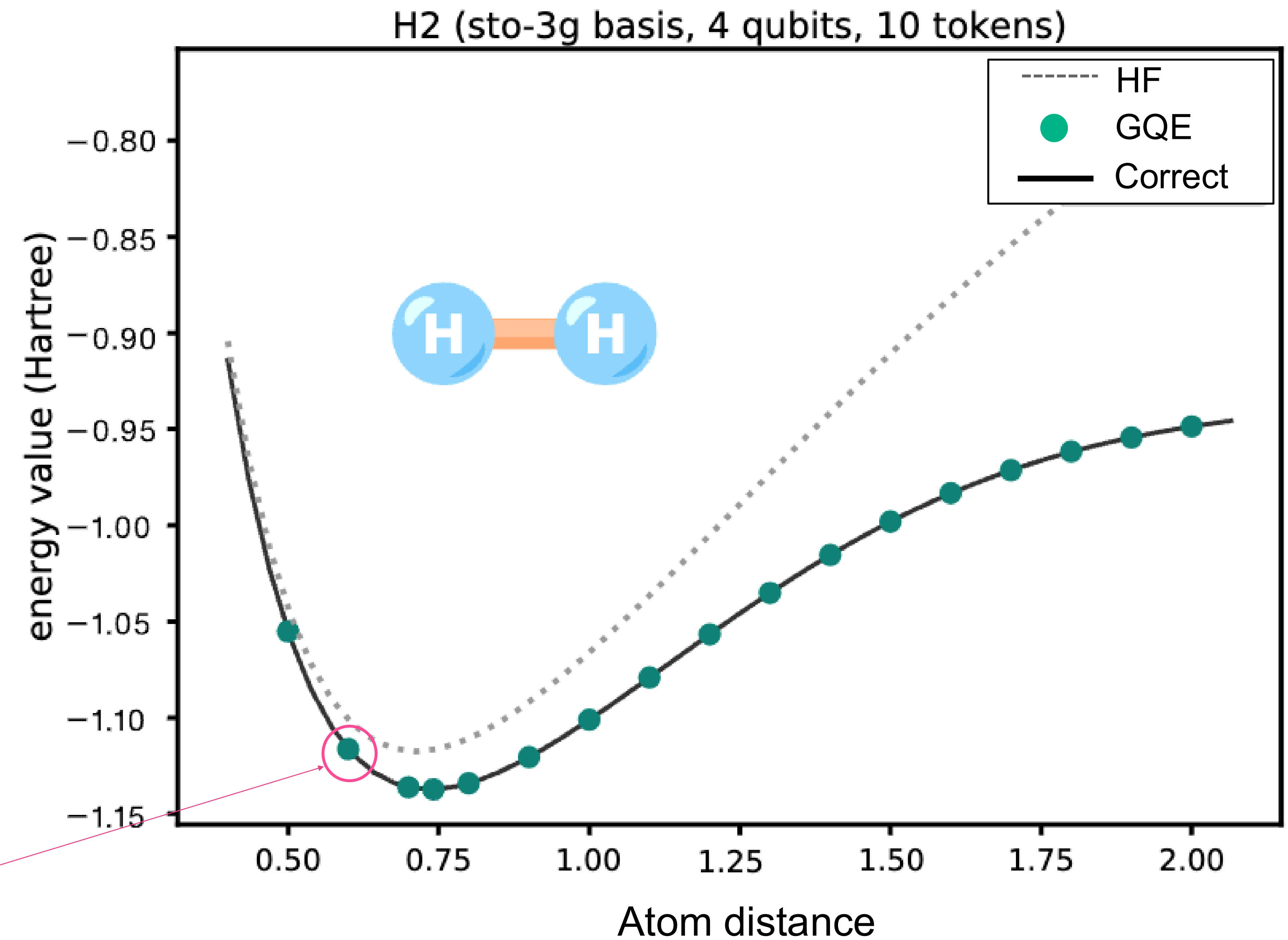
*

Operator pool:
Constructed from UCCSD operators



$$E(\vec{j}) := \langle \phi_0 | U^\dagger(\vec{j}) H U^\dagger(\vec{j}) | \phi_0 \rangle$$

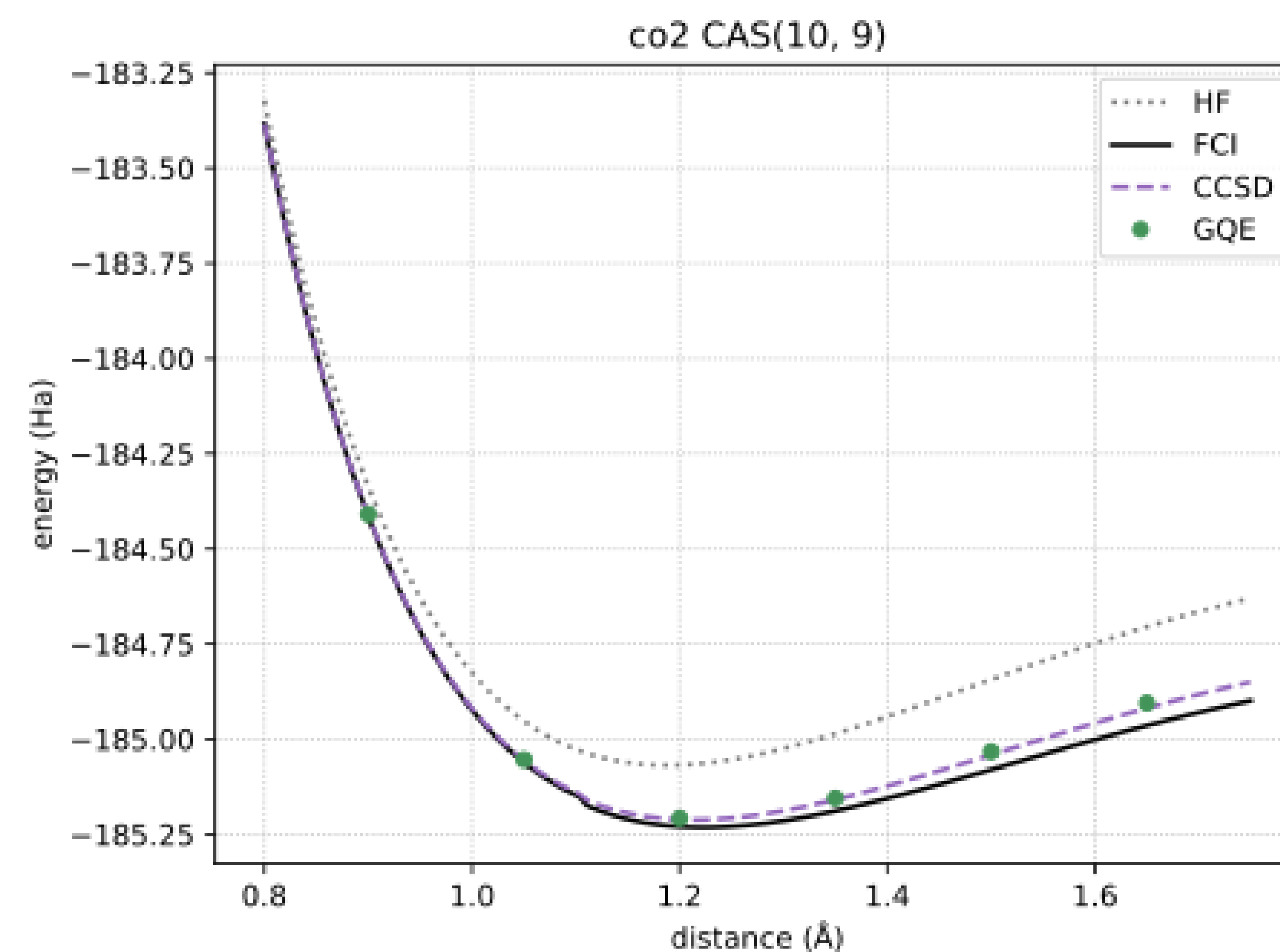
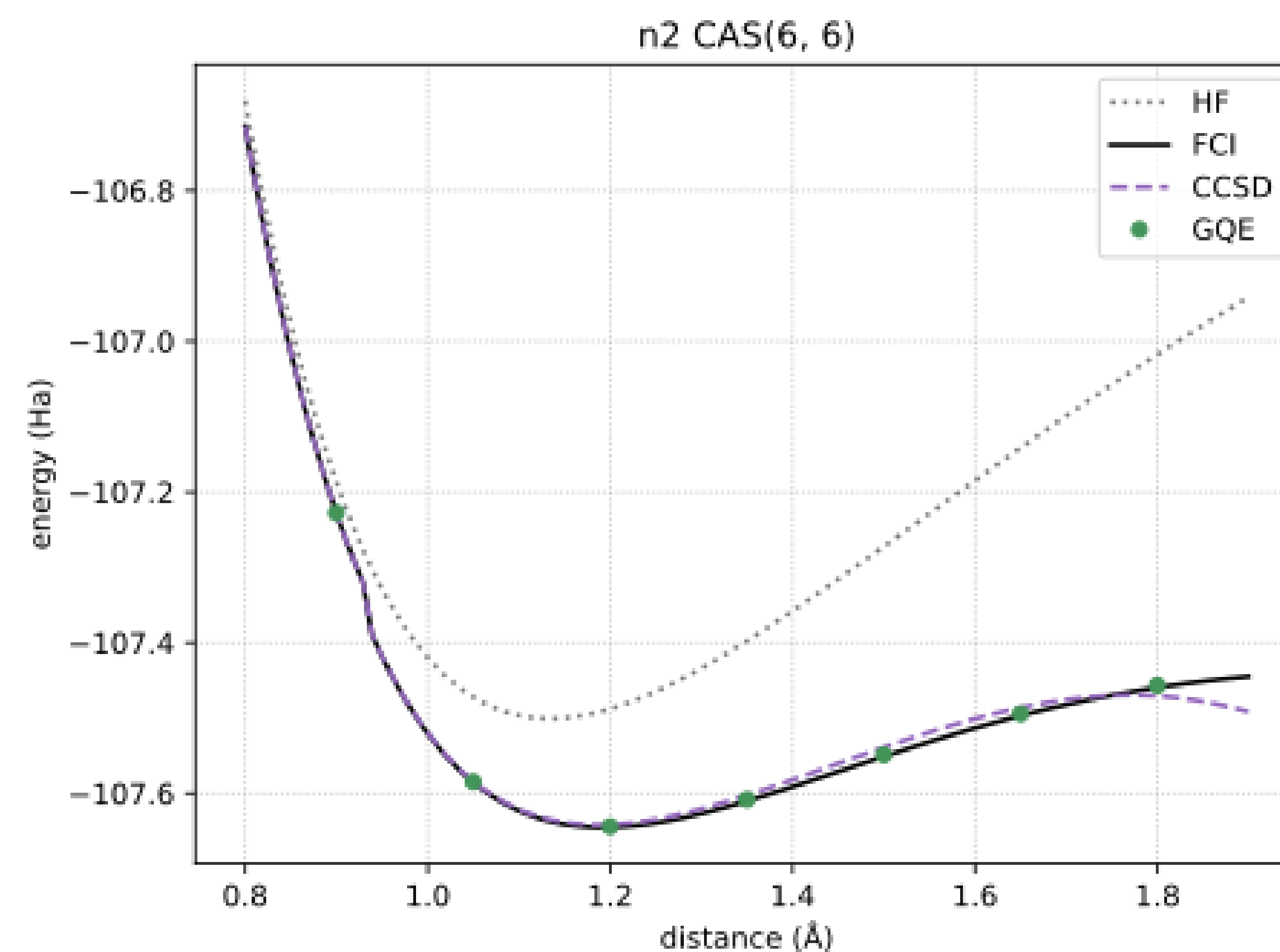
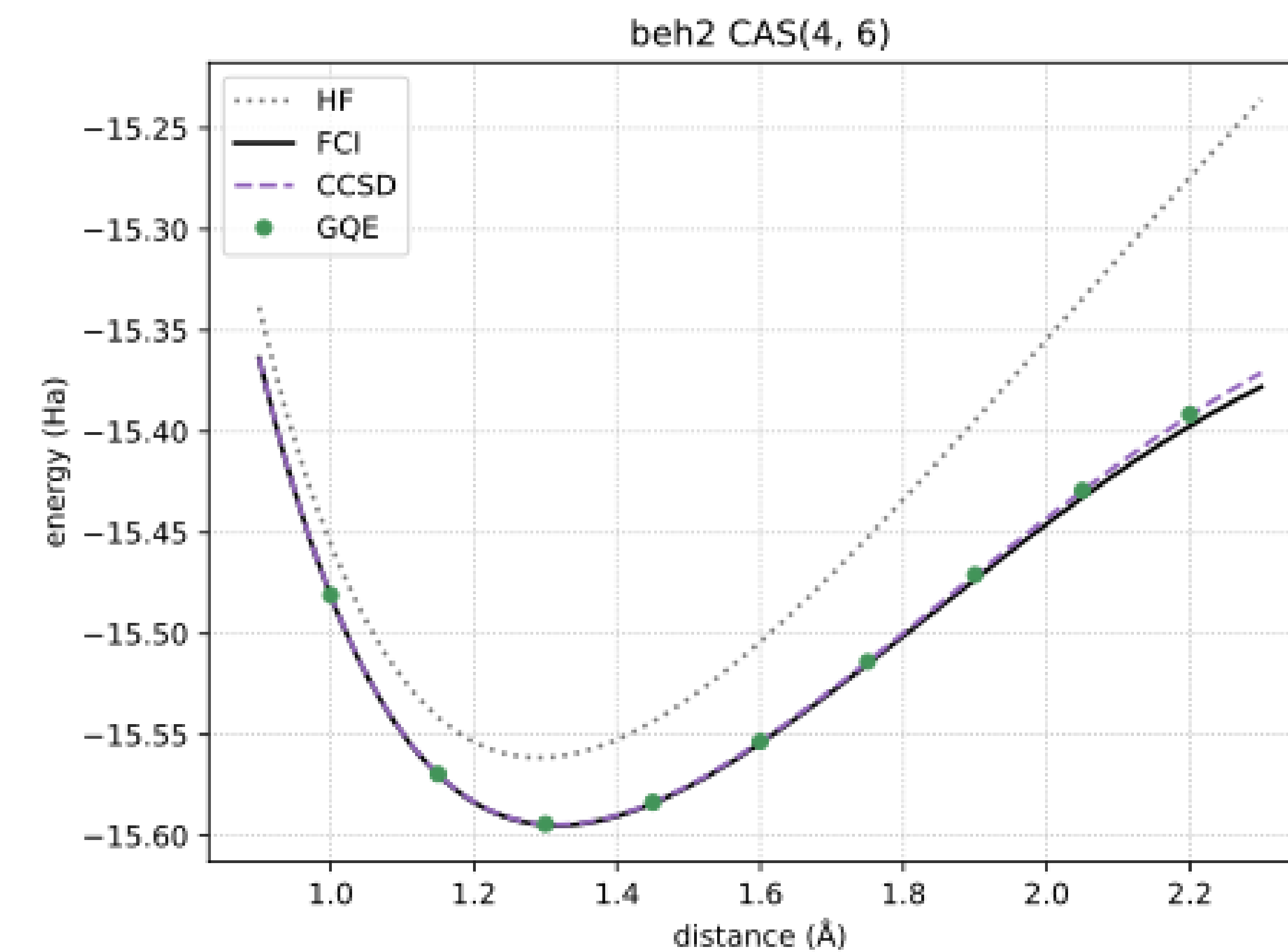
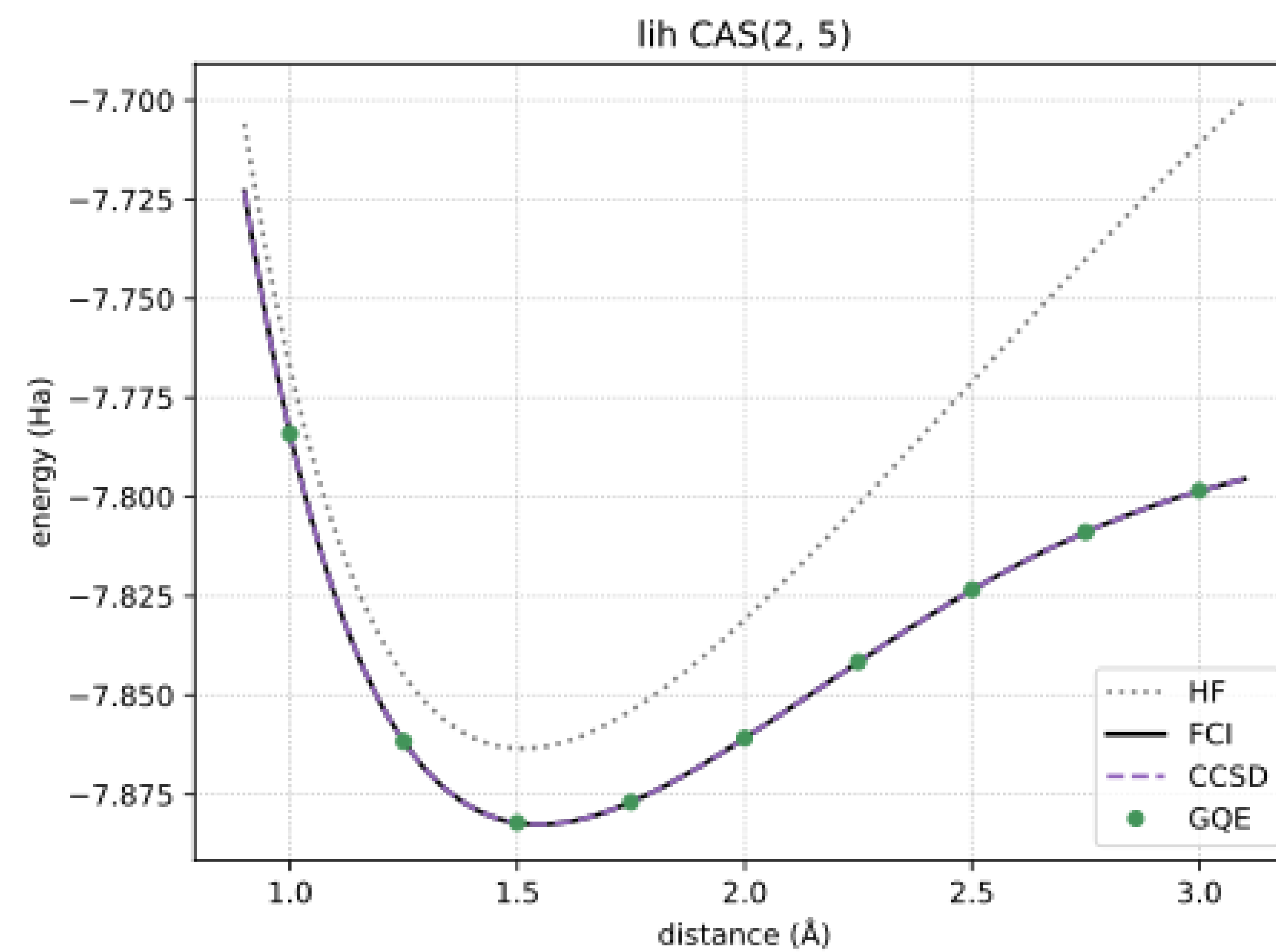
Ground state search result



*Having the form: $e^{iP_j t_j}$, where P_j is a Pauli operator, which appears when Trotterizing UCCSD operators. t_j is one of the discrete time.

Proof-of-concept: GPT-QE with chemistry Hamiltonians

Training works well (at the level of chemical accuracy)



Detailed comparison with other methods

GQE finds as good or better solutions than VQE with lower quantum computational overhead

12 qubits BeH2

12 qubits N2

18 qubits CO2

Depth

VQE: 92 operators, GQE: **60** operators

VQE: 117 operators, GQE: **100** operators

VQE: 560 operators, GQE: **200** operators

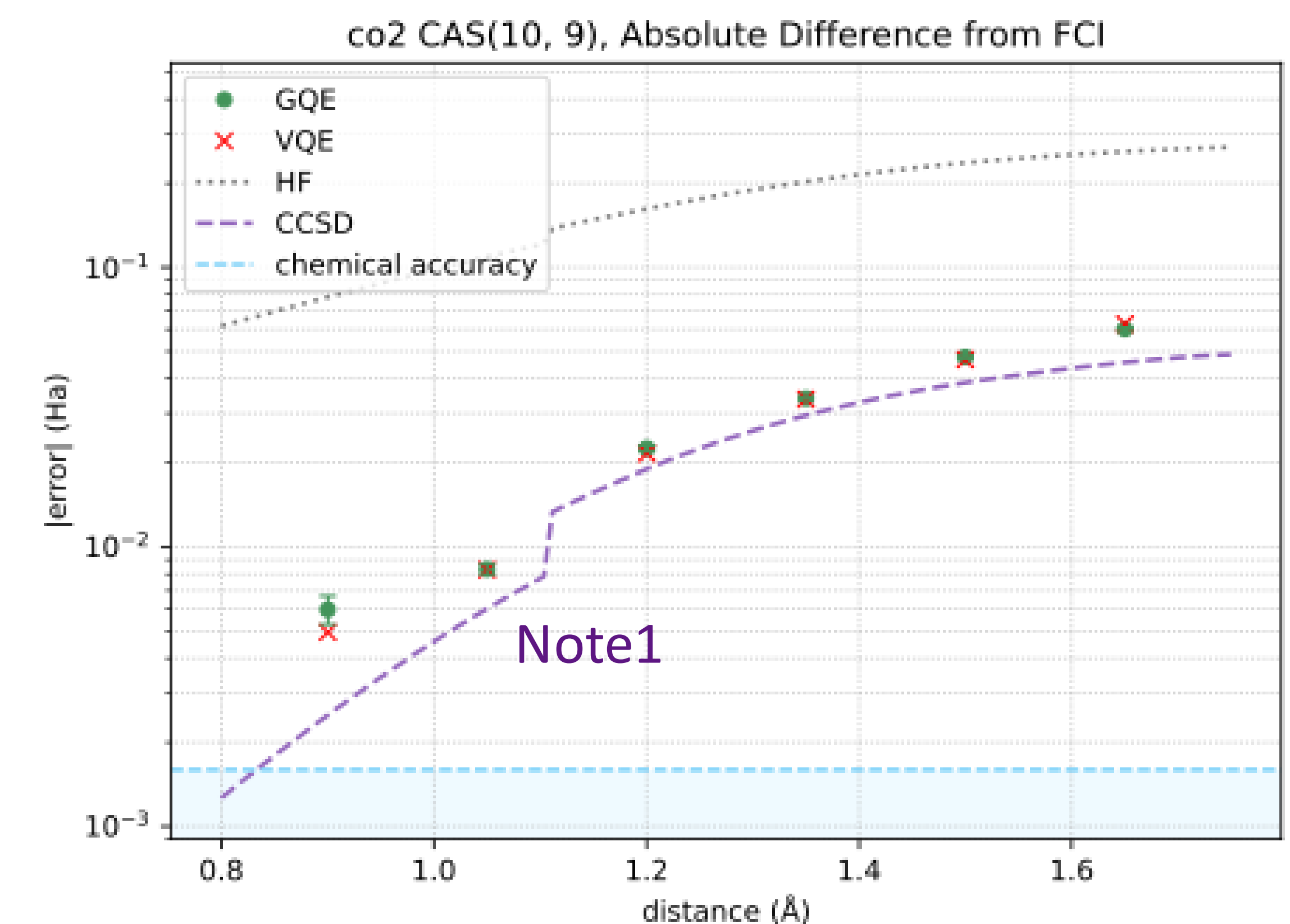
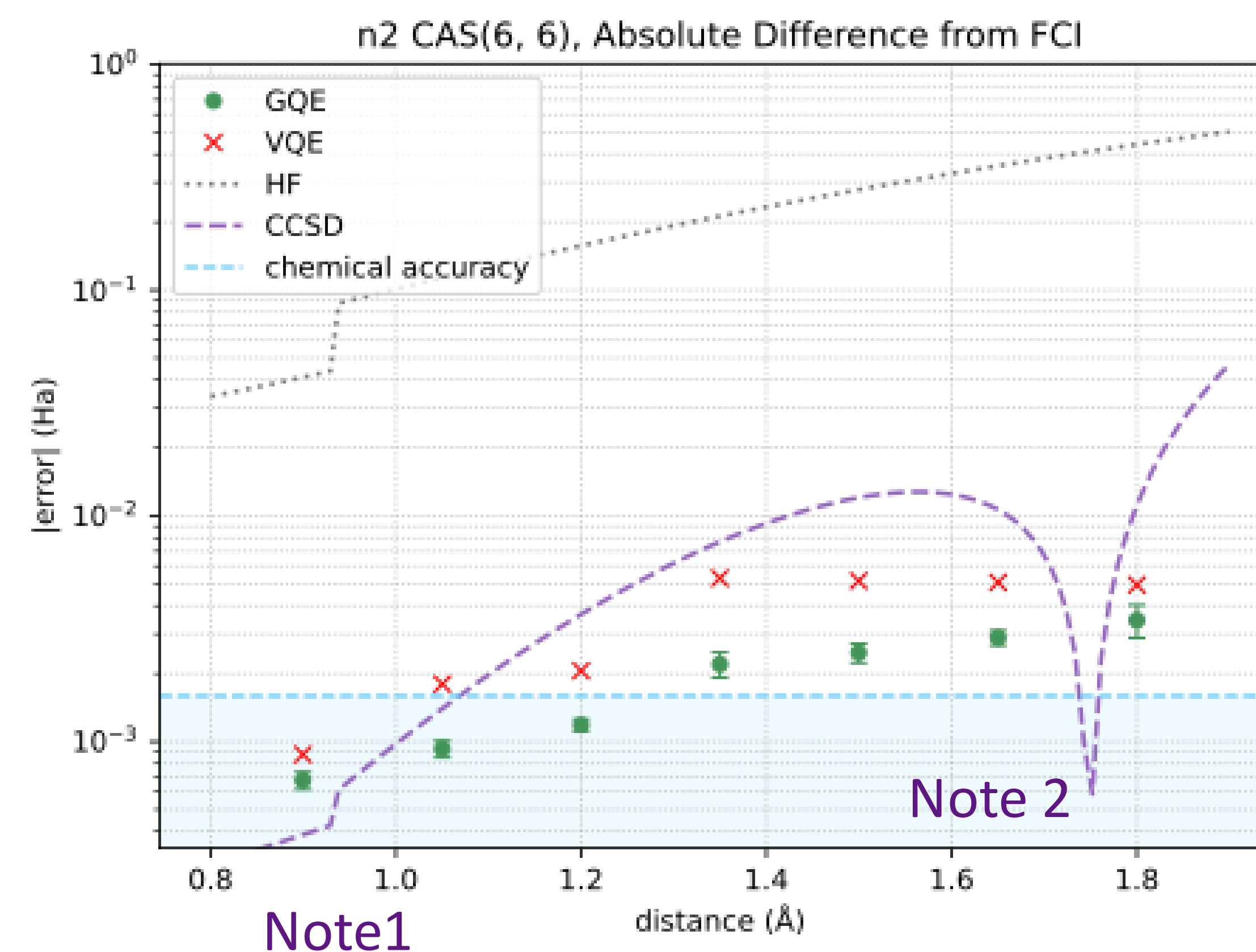
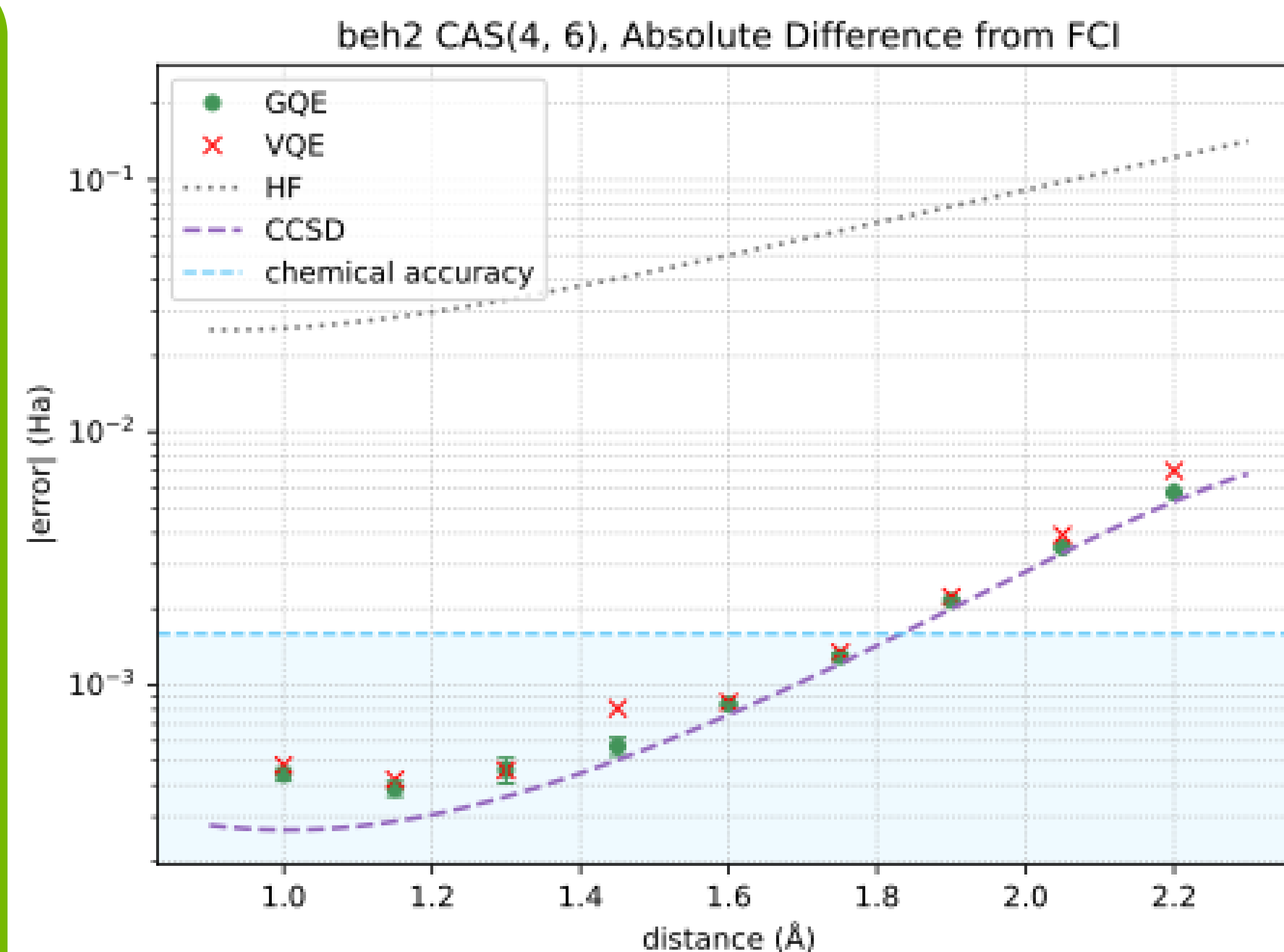
Energy
evaluation
count

VQE: ~400K
(2K epoch x (92 x 2)/epoch)
GQE: **~25K**
(0.5K epoch x 50/epoch)

VQE: ~750K
(3K epoch x (117 x 2)/epoch)
GQE: **~150K**
(3K epoch x 50/epoch)

VQE: ~10,000K
(10K epoch x (560 x 2)/epoch)
GQE: **~250K**
(5K epoch x 50/epoch)

Error
from
ground
truth



VQE: L-BFGS-B, UCCSD ansatz

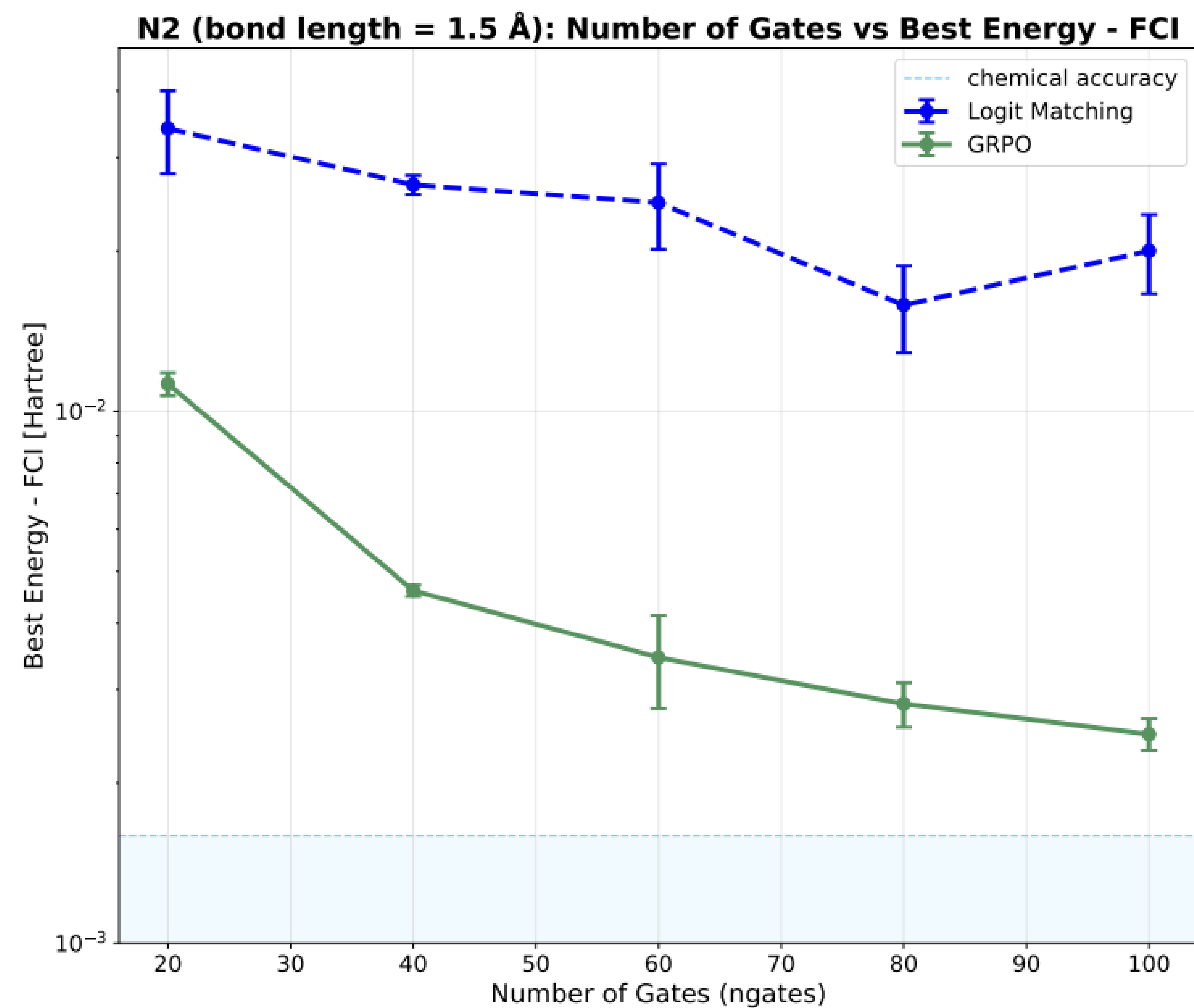
All data points are generated with CUDA-QX Solver

Note 1: Active space selection changes in the range (which equally affects all methods)

Note 2: The sign of the error in CCSD changes

Logit Matching vs GRPO

Leveraging advances in AI algorithms to improve performance



Group Relative Policy Optimization (GRPO) is an advanced algorithm used in reinforcement learning (RL) to fine-tune AI models.

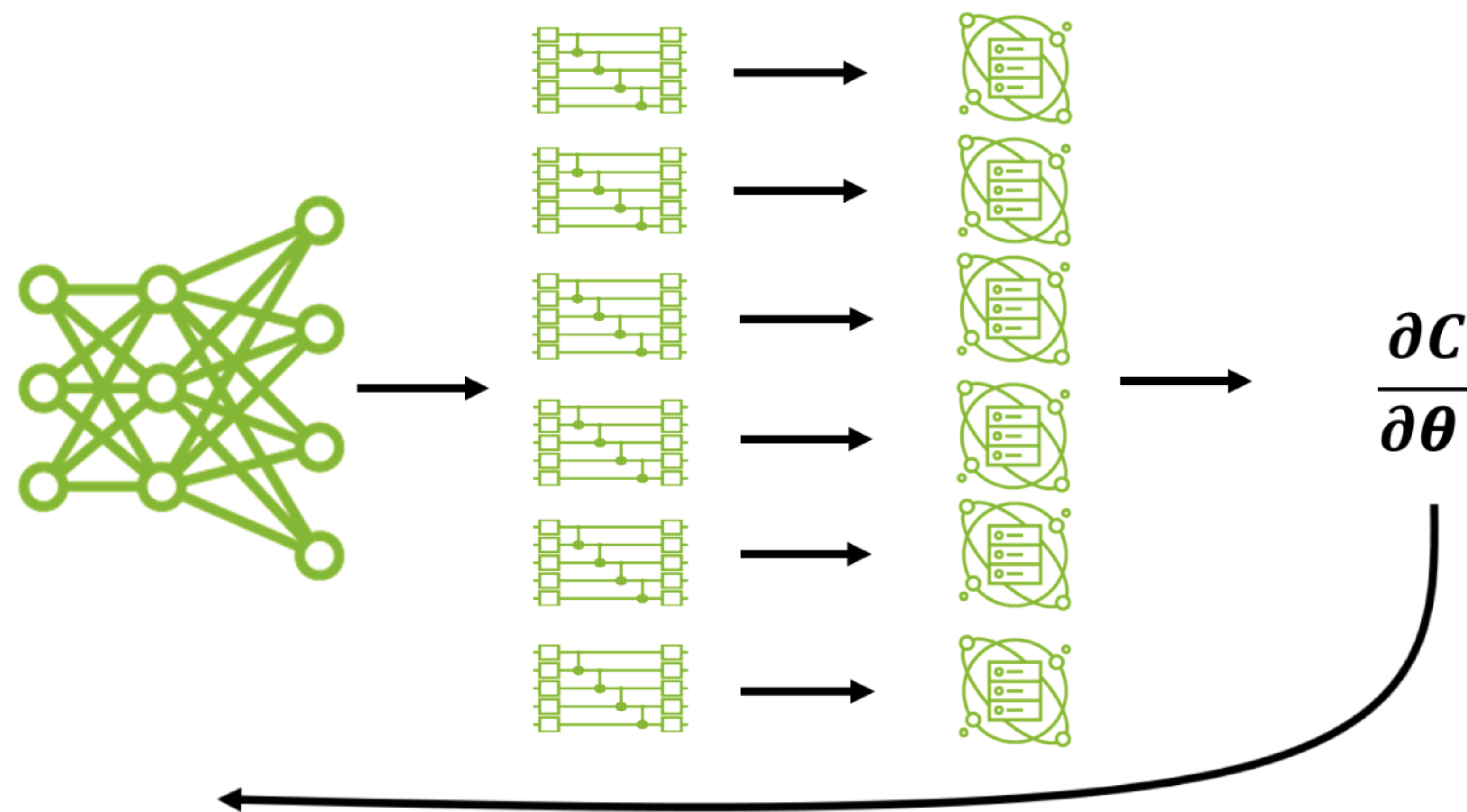
DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models.

The plot shows the best energy relative to the ground truth energy as a function of the number of gates (circuit depth).

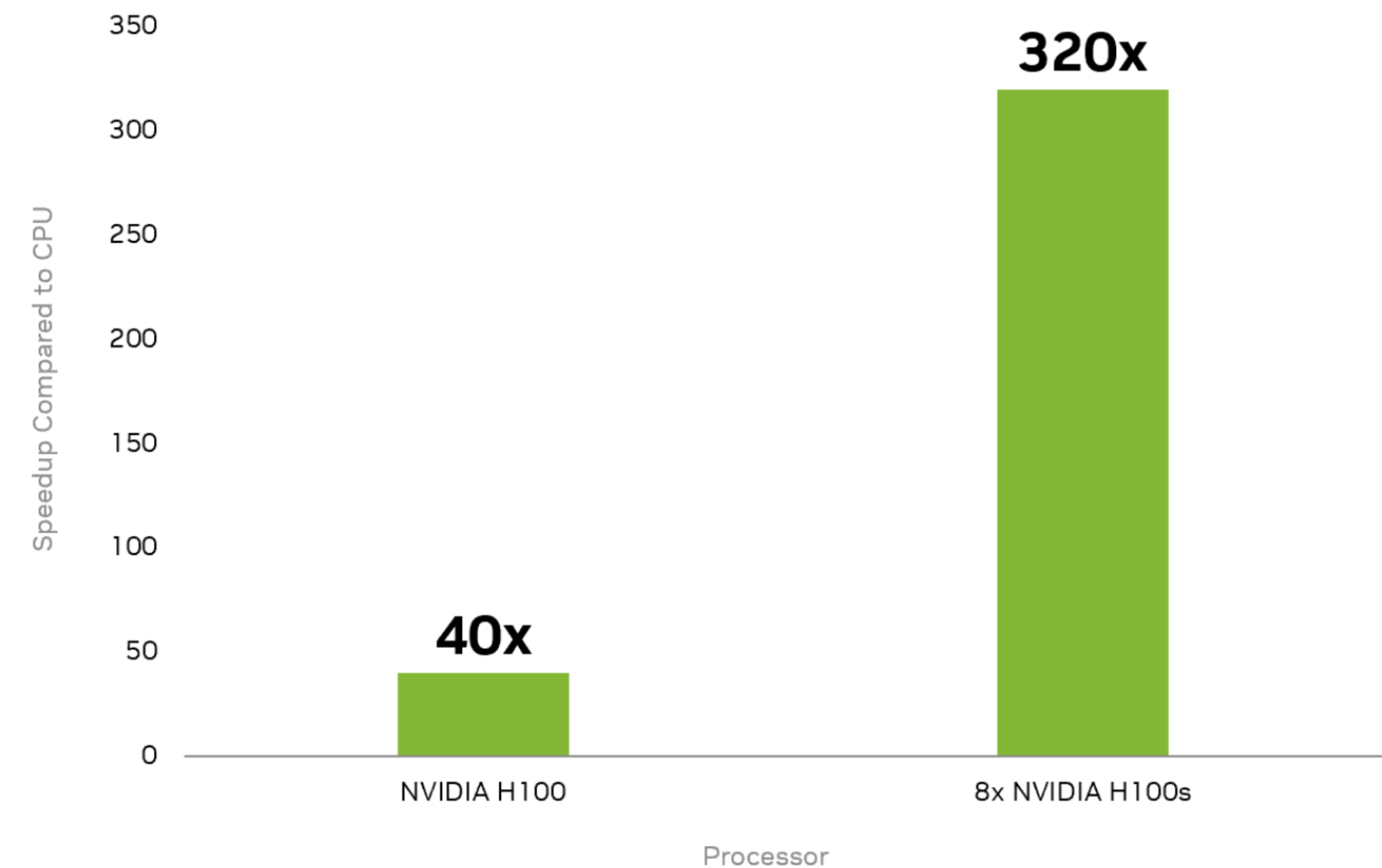
Parallelization Opportunities

Using CUDA-Q's `observe_async`

- Parallelizing the circuits in the batch at each epoch using the CUDA-Q's mQPU backend that supports the distributed model of computation.
- Asynchronous and parallel computational model.



Speedup Over CPU

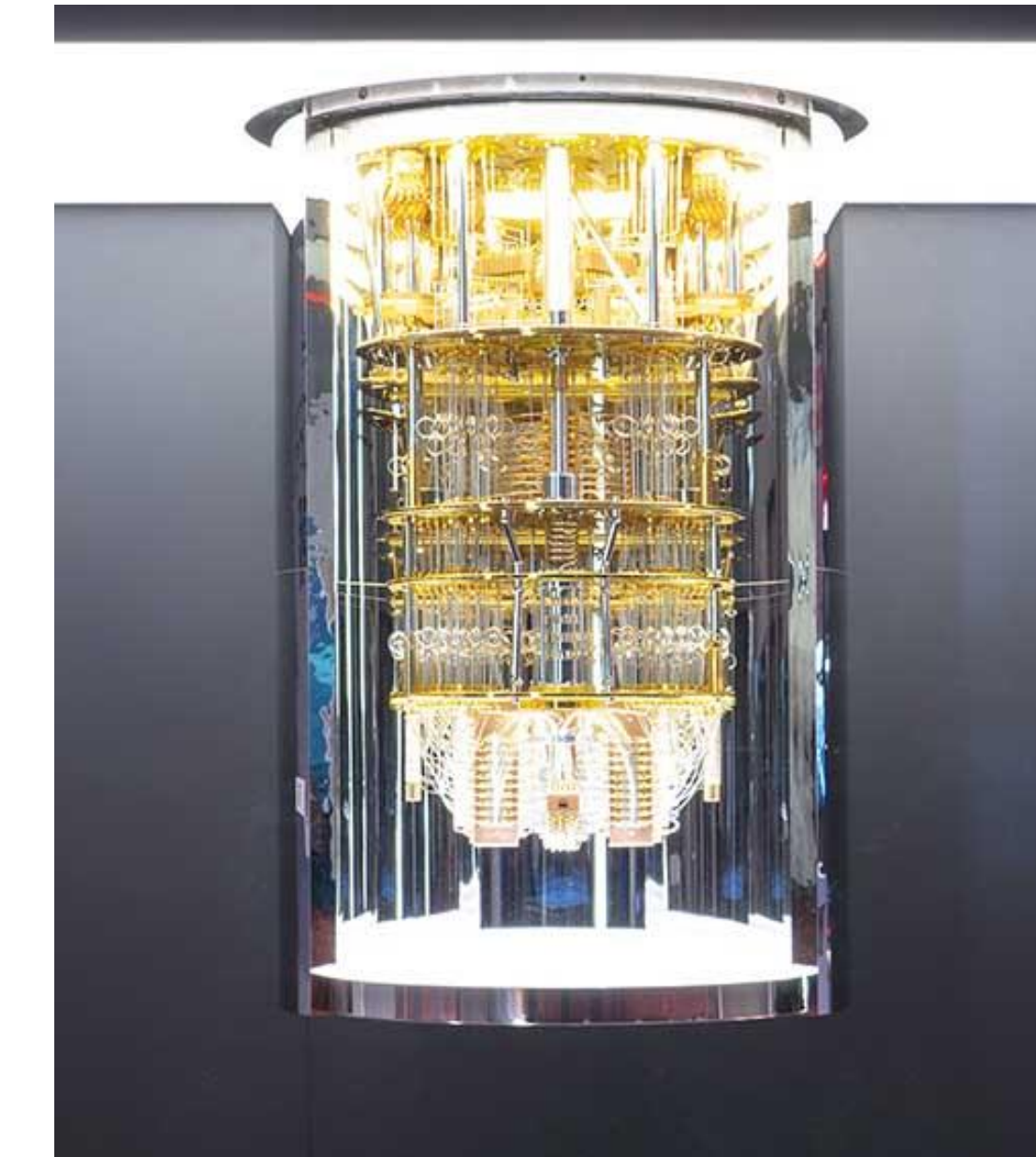
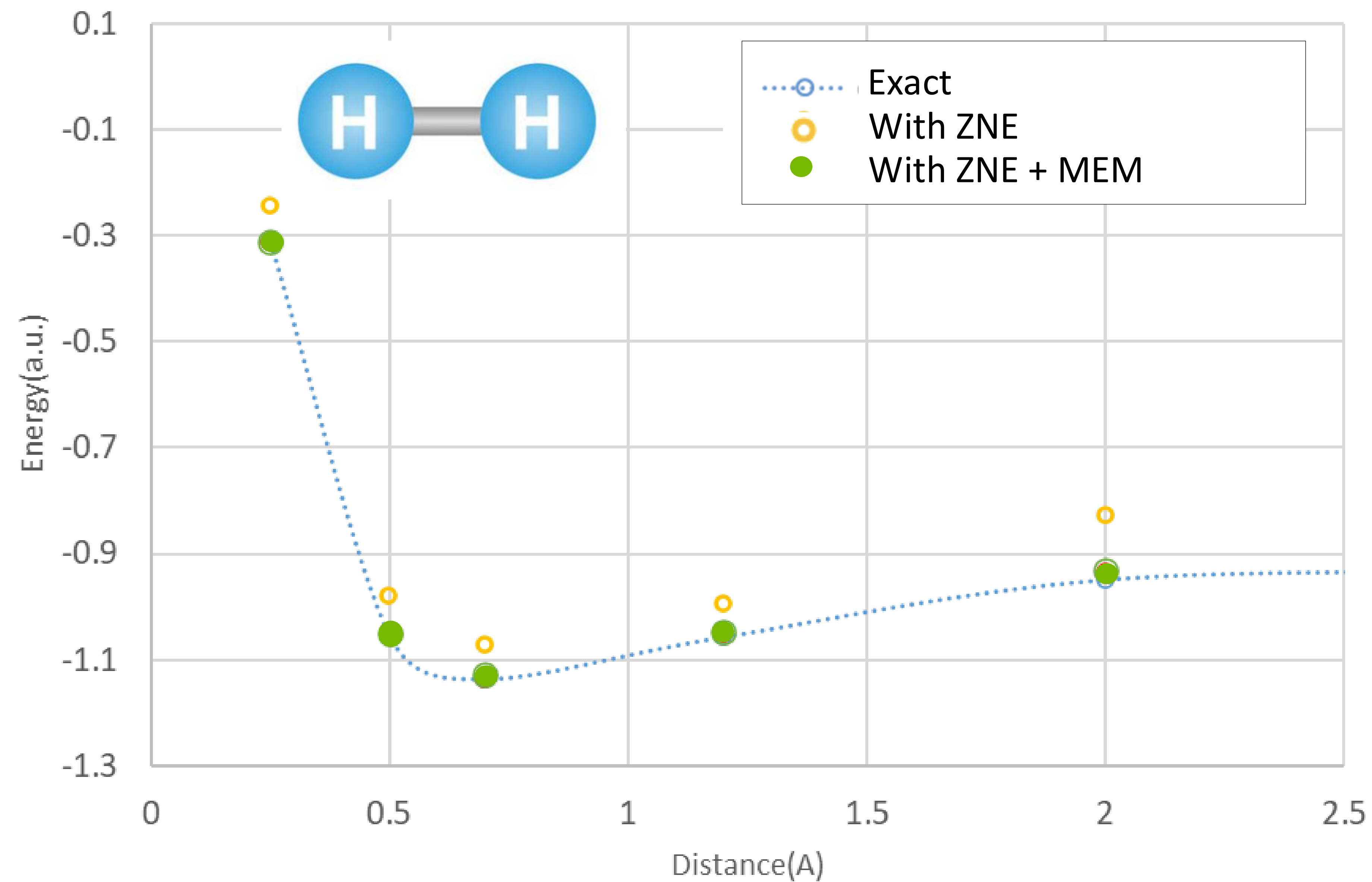


3.5 hours to train the 18-qubit CO₂ molecule using 8 H100 GPUs

GPT-QE in actual quantum device

Demonstration in actual quantum device

- GPT-QE in IBM device [4]:



IBM Kawasaki

- ZNE: zero noise extrapolation
- MEM: measurement error mitigation

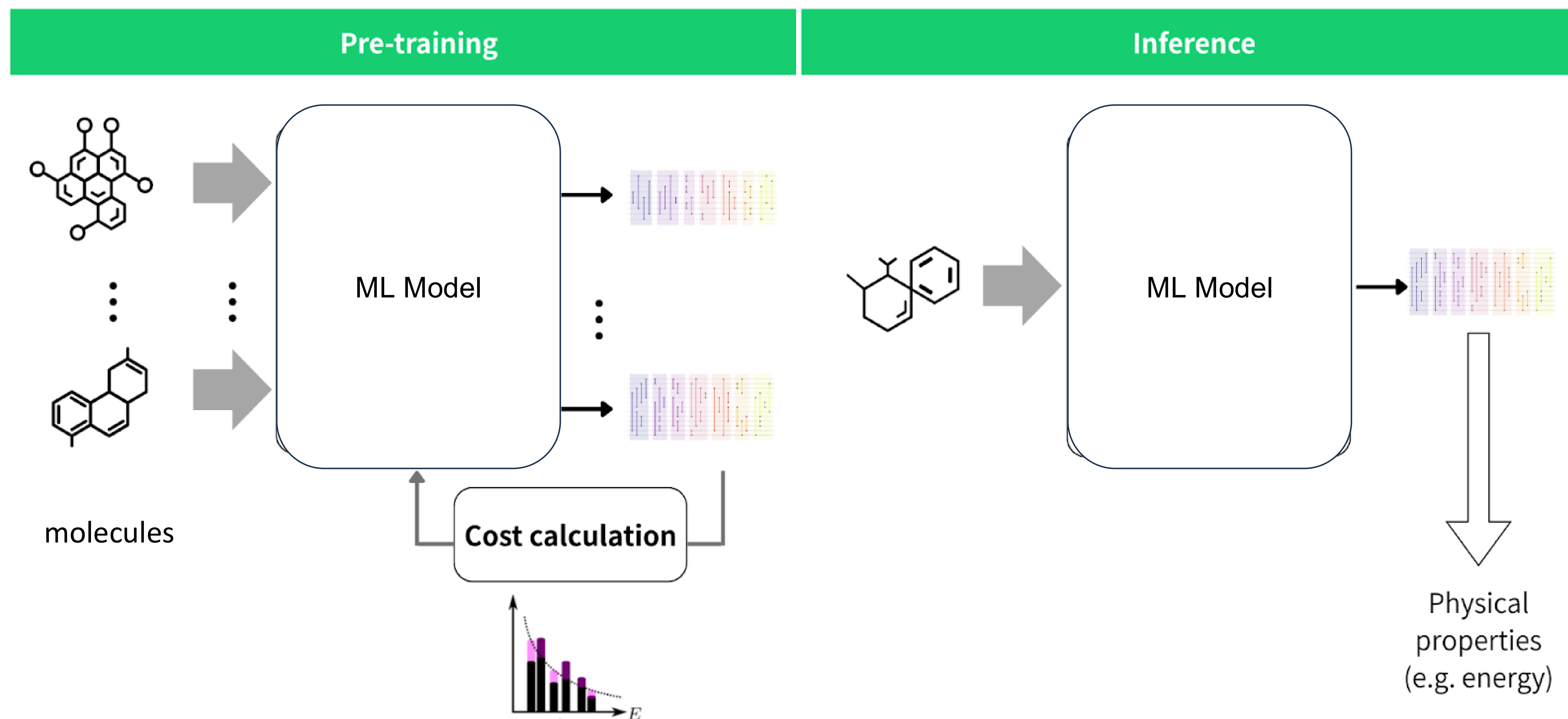
- Quantinuum announced that they successfully trained GPT-QE on their device [5]

[3] Follow-up work from Qi Gao (Mitsubishi Chemical) and Michihiko Sugawara (Keio Quantum Computing Center)

[4] <https://www.quantinuum.com/blog/genqai-a-new-era-at-the-quantum-ai-frontier>

Outlook: Unified ML model applicable for different molecules and geometries

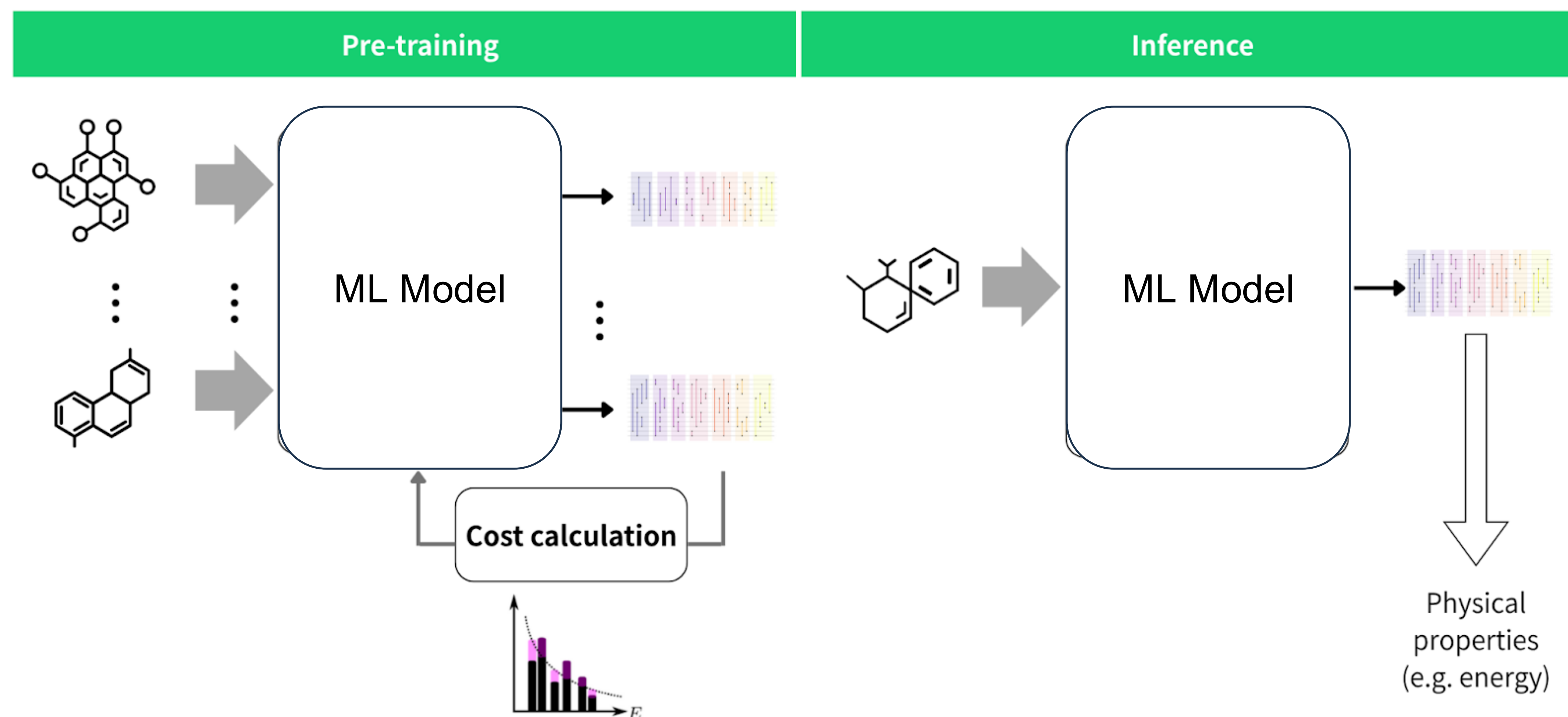
Molecule to molecule transfer



How to leverage the substantial structural and electronic similarities between related molecules without having to train from scratch for each new molecular system?

Outlook: Unified ML model applicable for different molecules and geometries

Molecule to molecule transfer



Steps to realize our ambition

- **Adding conditional input**
 - ➔ Already done for the combinatorial optimization problem [3] [7]
- **Scaling-up**
 - ➔ Using a classically synthesized dataset can be a good option [7]. We can also consider efficient training methods.
- **Embedding the info of molecules to the generative model**
 - There are a lot of work (e.g. MACE [8]) in AI for quantum chemistry to embed the molecular information into generative models.

[3] S. Minami, K. N., Y. Suzuki, A. Aspuru-Guzik & T. Kadowaki "Generative quantum combinatorial optimization by means of a novel conditional generative quantum eigensolver." *Digital Discovery*, 2025, 4, 2229-2243

[7] I. Tyagin, M. H. Farag, K. Sherbert, K. Shirali,, Y. Alexeev "QAOA-GPT: Efficient Generation of Adaptive and Regular Quantum Approximate Optimization Algorithm Circuits." , arXiv:2504.16350 (2025).

[8] Batatia, Ilyes, et al. "MACE: Higher order equivariant message passing neural networks for fast and accurate force fields." *Advances in neural information processing systems* 35 (2022): 11423-11436.

[9] Zhi Yin* et al., "SMILES-Inspired Transfer Learning for Quantum Operators in Generative Quantum Eigensolver" arXiv: 2509.19715

Summary

- Introduced the GQE algorithm which produces circuits optimized by classical generative models.
- The parameters embedded on the classical side as opposed to VQE where the quantum circuit is parameterized.
- Outlook : Embedding physics and other conditional inputs, transfer learning, pre-training, etc.